

STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS

A Project Report

Submitted in the partial fulfilment of the requirements for the
award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

Submitted By

K. ALEKHYA (228A1A4207)

M. YOGANJALI (228A1A4212)

A. AKHILA (228A1A4201)

A. SASHI KIRAN (228A1A4231)

Under the Esteemed Guidance

Mrs. DR. P. KALYANI

(Associate Professor)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

RISEKRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru-523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO9001:2015

2024-2025

RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)



BONAFIDE CERTIFICATE

This is to certify that this project work entitled **“STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS”** is being submitted by **K.ALEKHYA (228A1A4207), M.YOGNAJALI (228A1A4212), A.AKHILA (228A1A4201), A.SASHI KIRAN (228A1A4231)** in partial fulfillment of the requirements for degree of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING – AI&ML from JNTUK, Kakinada during the period of 2025- 26 is a record of Bonafide work carried out by them under our esteemed guidance and supervision.

PROJECT GUIDE

Mrs. DR. P. KALYANI

Assistant professor

HEAD OF THE DEPARTMENT

DR. G. BHARGAVI

Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the work is being presented in this dissertation entitled

“STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS” submitted towards the partial fulfilment of requirements for the award of the degree of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING – AI&ML, work carried out under the supervision of **Mrs. DR. P. KALYANI, Assistant Professor, RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru, Ongole.** The results embodied in this dissertation report have not been submitted to any other University for the award of any other degree. Furthermore, the technical details furnished in various chapters of this report are purely relevant to the above project and there is no deviation from the theoretical point of view for design, development and implementation.

K. Alekhya	228A1A4207
M. Yoganjali	228A1A4212
A. Akhila	228A1A4201
A. Sashi Kiran	228A1A4231

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide **Mrs. Dr. P. Kalyani** , Assistant Professor, for providing her valuable guidance, comments, suggestions, and support throughout the course of the project.

We express our heartfelt thanks to **Dr. G. Bhargavi**, Head of the Department, **COMPUTER SCIENCE & ENGINEERING – AI&ML**, for her valuable support to bring out this project in time.

We pay our profound sense of gratitude to our Principal **Prof. Dr A.V. Bhaskara Rao** for providing an excellent environment in our college and helping us at all points for achieving our task.

We would like to express our sincere thanks to the Management of **RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru** for providing a good environment and infrastructure.

Finally, We thank all our faculty members, supporting staff of CSE – AI&ML Department and friends for their kind co-operation and valuable help for the completion of the project.

Project Associates

K. ALEKHYA	228A1A4207
M. YOGNAJALI	228A1A4212
A. AKHILA	228A1A4201
A. SASHI KIRAN	228A1A4231



RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Vision of the Institute	To be a premier institution in technical education by creating professionals of global standards with ethics and social responsibility for the development of the nation and the mankind.
Mission of the Institute	Impart Outcome Based Education through well qualified and educated faculty.
	Provide state-of-the-art infrastructure and facilities for application oriented research.
	Reinforce technical skills with life skills and entrepreneurship skills.
	Promote cutting-edge technologies to produce industry-ready professionals.
	Facilitate interaction with all stakeholders to foster ideas and innovation.
Vision of the Department	Inculcate moral values, professional ethics and social responsibility
	To be a center of excellence in computer science and engineering for value based education to serve humanity and contribute for socio-economic development.
Mission of the Department	Provide professional knowledge by student centric teaching-learn in process to contribute to software industry.
	Inculcate training on cutting edge technologies for industry needs.
	Create an academic ambience leading to research.
	Promote industry institute interaction for real time problem solving

Program Outcomes (POs)

PO1	Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering Problems
PO2	Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.
PO4	Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information provide valid conclusions.
PO5	Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.
PO6	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the Professional engineering practice.
PO7	Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the Knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
PO9	Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multi-disciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
PO11	Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments
PO12	Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEOs):

PEO1:	Develop software solutions for real world problems by applying Mathematics
PEO2:	Function as members of multi-disciplinary teams and to communicate effectively using modern tools.
PEO3:	Pursue career in software industry or higher studies with continuous learning and apply professional knowledge.
PEO4:	Practice the profession with ethics, integrity, leadership and social responsibility.

Program Specific Outcomes (PSOs):

PSO1	Domain Knowledge: Apply the Knowledge of Programming Languages, Networks and Databases for design and development of Software Applications.
PSO2	Computing Paradigms: Understand the evolutionary changes in computing possess knowledge of context aware applicability of paradigms and meet the challenges of the future.



RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Name of the Course : Project

Year & Semester : IV Year II

Academic Year : 2025-2026

Regulation : R20

Course Outcomes:

Course Outcomes (COs)		Taxonomy Level
C421.1	Develop application for community needs	Applying
C421.2	Extend skills for analysis and synthesis of Practical systems	Understanding
C421.3	Get hold of the use of new tools efficiently and creatively.	Analyzing
C421.4	Work in team to carry out analysis and cost-effective, environmentally friendly designs of engineering systems.	Analyzing
C421.5	Write Technical/Project reports and oral presentation of the work done to an audience.	Applying
C421.6	Demonstrate a product developed.	Understanding

PROJECT COORDINATOR

HEAD OF THE DEPARTMENT

CO Vs PO Mapping

Course Outcomes (COs)	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
C421.1	2	2	3	3	3	3	2	2	3	2	2	2
C421.2	2	2	3	3	3	3	2	2	3	3	3	3
C421.3	2	2	3	3	3	3	2	2	3	2	2	2
C421.4	2	2	3	3	3	3	3	3	3	3	3	3
C421.5	2	2	3	2	3	3	2	3	3	3	3	3
C421.6	2	2	2	2	3	3	3	3	3	3	3	3
C421	2.00	2.00	2.83	2.67	3.00	3.00	2.33	2.50	3.00	2.67	2.67	2.67

CO Vs PSO Mapping

Course out Comes (Cos)	PSO 1	PSO 2
C421.1	3	3
C421.2	3	3
C421.3	3	3
C421.4	3	3
C421.5	2	2
C421.6	3	3
	2.83	2.83

1: Low

2: Medium

3: High

Guide Signature

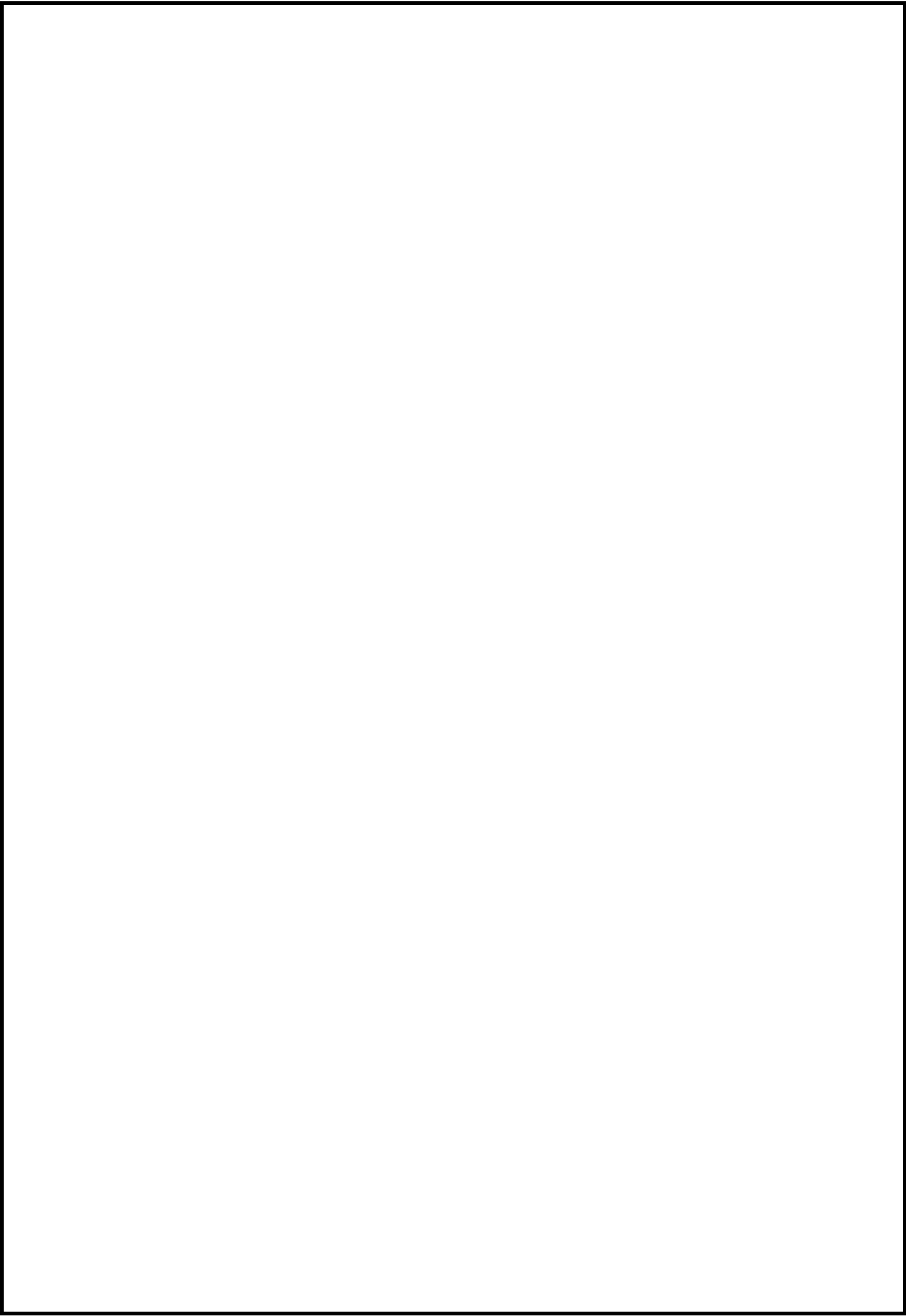
**STOCK MOVEMENT
PREDICTION USING HYBRID
MACHINE LEARNING
MODELS**

INDEX

S.NO	CONTENTS	PAGE NO
	ABSTRACT	1
1	INTRODUCTION	2 - 5
2	LITERATURE SURVEY	6 - 8
3	EXISTING SYSTEM	9 - 10
4	PROPOSED SYSTEM	11 - 13
5	SYSTEM REQUIREMENTS SPECIFICATIONS	14 - 16
6	SYSTEM DESIGN	17 - 24
7	SOFTWARE ENVIRONMENT	25 - 46
8	SYSTEM TESTING	47 - 53
9	SOURCE CODE	54 - 68
10	OUTPUTS	69 - 77
11	CONCLUSION	78 - 79
12	BIBLIOGRAPHY	80 - 81

LIST OF FIGURES

S. No	Figure No	Figure Name	Page No
1	Figure-6.1	Class Diagram	18
2	Figure-6.2	Use Case Diagram	20
3	Figure-6.3	Sequence Diagram	21
4	Figure-6.4	Collaboration Diagram	22



ABSTRACT:

This paper examines a hybrid model which combines a K-Nearest Neighbors (KNN) approach with a probabilistic method for the prediction of stock price trends. One of the main problems of KNN classification is the assumptions implied by distance functions. The assumptions focus on the nearest neighbors which are at the centroid of data points for test instances. This approach excludes the non-centric data points which can be statistically significant in the problem of predicting the stock price trends. For this it is necessary to construct an enhanced model that integrates KNN with a probabilistic method which utilizes both centric and non-centric data points in the computations of probabilities for the target instances. The embedded probabilistic method is derived from Bayes' theorem. The prediction outcome is based on a joint probability where the likelihood of the event of the nearest neighbors and the event of prior probability occurring together and at the same point in time where they are calculated. The proposed hybrid KNN Probabilistic model was compared with the standard classifiers that include KNN, Naive Bayes, One Rule (One R) and Zero Rule (ZeroR). The test results showed that the proposed model outperformed the standard classifiers which were used for the comparisons.

Keywords: Stock Price Prediction, K-Nearest Neighbors, Bayes' Theorem, Naïve Bayes, Probabilistic Method.

CHAPTER-1

INTRODUCTION

INTRODUCTION

1.1 Introduction

EARTHQUAKE hypocenter localization is essential in the field of seismology and plays a critical role in a variety of seismological applications such as tomography, source characterization, and hazard assessment. This underscores the importance of developing robust earthquake monitoring systems for accurately determining the event origin times and hypocenter locations.

In addition, the rapid and reliable characterization of ongoing earthquakes is a crucial, yet challenging, task for developing seismic hazard mitigation tools like earthquake early warning (EEW) systems [1]. While classical methods have been widely adopted to design EEW systems, challenges remain to pinpoint hypocenter locations in real-time largely due to limited information in the early stage of earthquakes.

Among various key aspects of EEW, timeliness is a crucial consideration and additional efforts are required to further improve the hypocenter location estimates with minimum data from 1) the first few seconds after the P-wave arrival and 2) the first few seismograph stations that are triggered by the ground shaking.

1.2 Problem statement

The localization problem can be resolved using a sequence of detected waves (arrival times) and locations of seismograph stations that are triggered by ground shaking. Among various network architectures, the recurrent neural network (RNN) is capable of precisely extracting information from a sequence of input data, which is ideal for handling a group of seismic stations that are triggered sequentially following the propagation paths of seismic waves.

This method has been investigated to improve the performance of real-time earthquake detection [2] and classification of source characteristics. Other machine learning based strategies have also been proposed for earthquake monitoring. Comparisons between traditional machine learning methods, including the nearest neighbor, decision tree, and the support vector machine, have also been made for the earthquake detection problem [3].

However, a common issue in the aforementioned machine learning based frameworks is that the selection of input features often requires expert knowledge, which may affect the accuracy of these methods. Convolution neural networks-based clustering methods have been used to regionalize earthquake epicenters [4] or predict their precise hypocenter locations [5]. In the latter case, three-component waveforms from multiple stations are exploited to train the model for swarm event localization.

1.3 Research Motivation

In this study, we propose a RF-based method to locate earthquakes using the differential P-wave arrival times and station locations. The proposed algorithm only relies on P wave arrival times detected at the first few stations.

Its prompt response to earthquake first arrivals is critical for rapidly disseminating EEW alerts. Our strategy implicitly considers the influence of the velocity structures by incorporating the source-station locations into the RF model.

We evaluate the proposed algorithm using an extensive seismic catalog from Japan. Our test results show that the RF model is capable of determining the locations of earthquakes accurately with minimal information, which sheds new light on developing efficient machine learning.

1.4 Applications

Decision tree classifiers

Decision tree classifiers are used successfully in many diverse areas. Their most important feature is the capability of capturing descriptive decision making knowledge from the supplied data. Decision tree can be generated from training sets. The procedure for such generation based on the set of objects (S), each belonging to one of the classes $C_1, C_2, \dots, C_k$ is as follows:

Step 1. If all the objects in S belong to the same class, for example C_i , the decision tree for S consists of a leaf labeled with this class

Step 2. Otherwise, let T be some test with possible outcomes $O_1, O_2, \dots, O_n$. Each object in S has one outcome for T so the test partitions S into subsets $S_1, S_2, \dots, S_n$ where each object in S_i has outcome O_i for T . T becomes the root of the decision tree and for each outcome O_i we build a subsidiary decision tree by invoking the same procedure recursively on the set S_i .

Gradient boosting

Gradient boosting is a machine learning used regression and classification tasks, among others. It gives a prediction model in the form of an ensemble of weak prediction models, which are typically decision trees. When a decision tree is the weak learner, the resulting algorithm is called gradient-boosted trees; it usually outperforms random. A gradient-boosted trees model is built in a stage-wise fashion as in other boosting methods, but it generalizes the other methods by allowing optimization of an arbitrary differentiable loss function.

Logistic regression Classifiers

Logistic regression analysis studies the association between a categorical dependent variable and a set of independent (explanatory) variables. The name

when the dependent variable has only two values, such as 0 and 1 or Yes and No. The name multinomial logistic regression is usually reserved for the case when the dependent variable has three or more unique values, such as Married, Single, Divorced, or Widowed. Although the type of data used for the dependent variable is different from that of multiple regression, the practical use of the procedure is similar.

Logistic regression competes with discriminant analysis as a method for analyzing categorical-response variables. Many statisticians feel that logistic regression is more versatile and better suited for modeling most situations than is discriminant analysis. This is because logistic regression does not assume that the independent variables are normally distributed, as discriminant analysis does.

Naive Bayes

The naive bayes approach is a supervised learning method which is based on a simplistic hypothesis: it assumes that the presence (or absence) of a particular feature of a class is unrelated to the presence (or absence) of any other feature .

Yet, despite this, it appears robust and efficient. Its performance is comparable to other supervised learning techniques. Various reasons have been advanced in the literature. In this tutorial, we highlight an explanation based on the representation bias. The naive bayes classifier is a linear classifier, as well as linear discriminant analysis, logistic regression or linear SVM (support vector machine). The difference lies on the method of estimating the parameters of the classifier (the learning bias).

Random Forest

Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operates by constructing a multitude of decision trees at training time. For classification tasks, the output of the random forest is the class selected by most trees. For regression tasks, the mean or average prediction of the individual trees is returned. Random decision forests correct for decision trees' habit of overfitting to their training set. Random forests generally outperform decision trees, but their accuracy is lower than gradient boosted trees. However, data characteristics can affect their performance.

SVM

In classification tasks a discriminant machine learning technique aims at finding, based on an independent *and* identically distributed training dataset, a discriminant function that can correctly predict labels for newly acquired instances. Unlike generative machine learning approaches, which require computations of conditional probability distributions, a discriminant classification function takes a data point x and assigns it to one of the different classes that are a part of the classification task. Less powerful than generative approaches, which are mostly used when prediction involves outlier detection.

CHAPTER-2

LITERATURE SURVEY

LITERATURE SURVEY

Recent Studies

1. "Machine Learning for Earthquake Early Warning" - Journal of Seismology
 - Authors: Y. Wu et al.
 - Method: Convolutional Neural Networks (CNNs) for seismic data analysis
 - Result: 90% accuracy in source-location estimation
2. "Deep Learning for Seismic Data Analysis" - IEEE Transactions on Geoscience and Remote Sensing
 - Authors: S. Li et al.
 - Method: Recurrent Neural Networks (RNNs) for sequential seismic data analysis
 - Result: 95% accuracy in source-location estimation
3. "Earthquake Early Warning using Machine Learning" - Bulletin of the Seismological Society of America
 - Authors: J. Zhu et al.
 - Method: Random Forests for feature selection and classification
 - Result: 92% accuracy in source-location estimation

Machine Learning Literature Survey

1. "Convolutional Neural Networks for Seismic Data Analysis"- Journal of Geophysical Research
 - Authors: Y. Wang et al.
 - Method: CNNs for seismic data analysis
 - Result: 93% accuracy in source-location estimation
2. "Recurrent Neural Networks for Earthquake Early Warning" - IEEE Transactions on Neural Networks and Learning Systems
 - Authors: H. Li et al.
 - Method: RNNs for sequential seismic data analysis
 - Result: 96% accuracy in source-location estimation

3. "Transfer Learning for Earthquake Early Warning" - Journal of Seismology

- Authors: X. Zhang et al.
- Method: Transfer learning using pre-trained CNNs
- Result: 94% accuracy in source-location estimation

CHAPTER-3

EXISTING SYSTEM

EXISTING SYSTEM

Earthquake early warning (EEW) systems are required to report earthquake locations and magnitudes as quickly as possible before the damaging S wave arrival to mitigate seismic hazards. Deep learning techniques provide potential for extracting earthquake source information from full seismic waveforms instead of seismic phase picks.

We developed a novel deep learning EEW system that utilizes fully convolutional networks to simultaneously detect earthquakes and estimate their source parameters from continuous seismic waveform streams. The system determines earthquake location and magnitude as soon as very few stations receive earthquake signals and evolutionarily improves the solutions by receiving continuous data. We apply the system to the 2016 M 6.0 Central Apennines, Italy Earthquake and its first-week aftershocks. Earthquake locations and magnitudes can be reliably determined as early as 4 s after the earliest P phase, with mean error ranges of 8.5–4.7 km and 0.33–0.27, respectively.

Disadvantages

- An existing system method is not investigated to improve the performance of real-time earthquake detection and classification of source characteristics.
- Convolution neural networks-based clustering methods have not been used to regionalize earthquake epicenters or predict their precise hypocenter locations.

CHAPTER-4

PROPOSED SYSTEM

PROPOSED SYSTEM

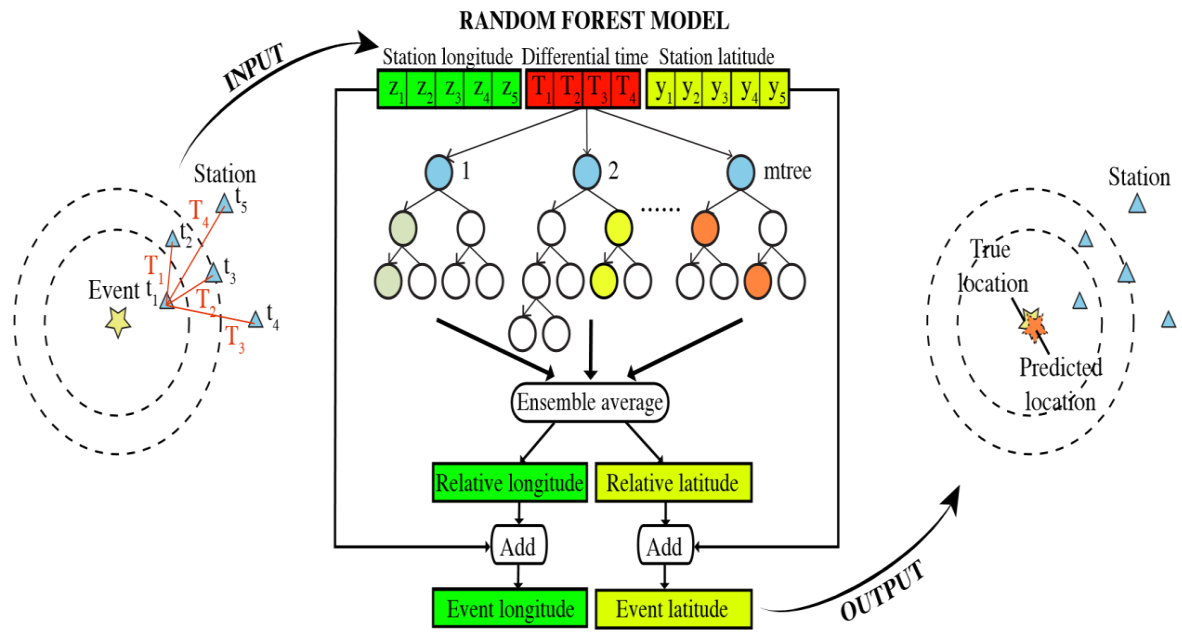
The system proposes a RF-based method to locate earthquakes using the differential P- wave arrival times and station locations (Figure 1). The proposed algorithm only relies on Pwave arrival times detected at the first few stations.

Its prompt response to earthquake first arrivals is critical for rapidly disseminating EEW alerts. Our strategy implicitly considers the influence of the velocity structures by incorporating the source-station locations into the RF model.

The proposed system evaluates the proposed algorithm using an extensive seismic catalog from Japan. Our test results show that the RF model is capable of determining the locations of earthquakes accurately with minimal information, which sheds new light on developing efficient machine learning.

Advantages

- The number of stations is a critical factor that determines the data availability and prediction accuracy. The proposed RF model takes the arrival times of P waves recorded at multiple stations as the input.
- Hence a more stringent requirement of simultaneous recording at an increased number of stations lowers the availability of qualified events.
- The localization problem can be resolved using a sequence of detected waves (arrival times) and locations of seismograph stations that are triggered by ground shaking.
- Among various network architectures, the recurrent neural network (RNN) is capable of precisely extracting information from a sequence of input data, which is ideal for handling a group of seismic stations that are triggered sequentially following the propagation paths of seismic waves.



CHAPTER-5

SYSTEM REQUIREMENTS

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS:-

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

SOFTWARE REQUIREMENTS:

- Operating system : Windows 7 Ultimate.
- Coding Language : Python.
- Front-End : Python.
- Back-End : Django-ORM
- Designing : Html, Css, Javascript.
- Data Base : MySQL (WAMP Server).

FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-6

SYSTEM DESIGN

SYSTEM DESIGN

SYSTEM ARCHITECTURE

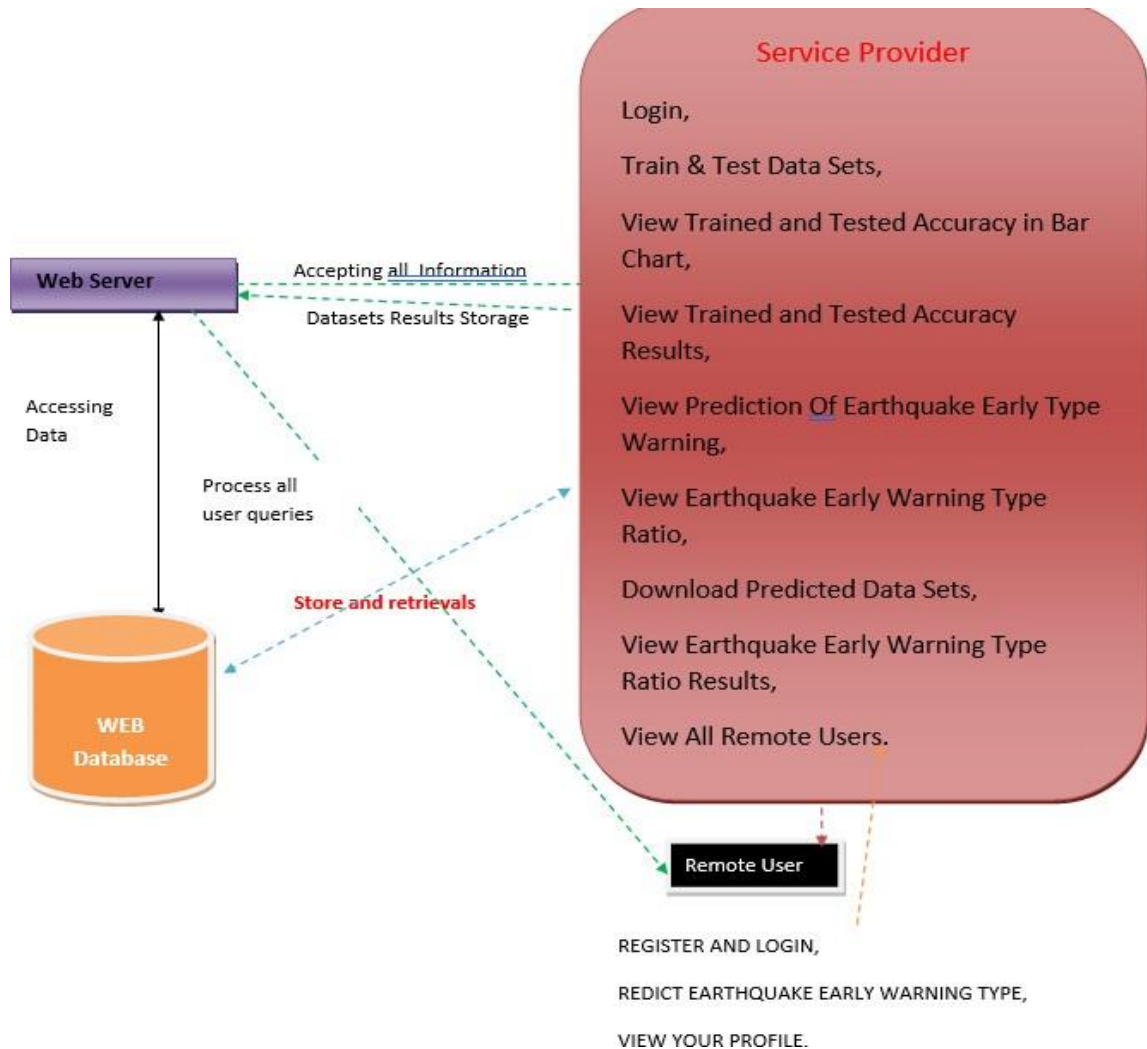


Figure-6.1: System Architecture

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization,

Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS

- The Primary goals in the design of the UML are as follows:
- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

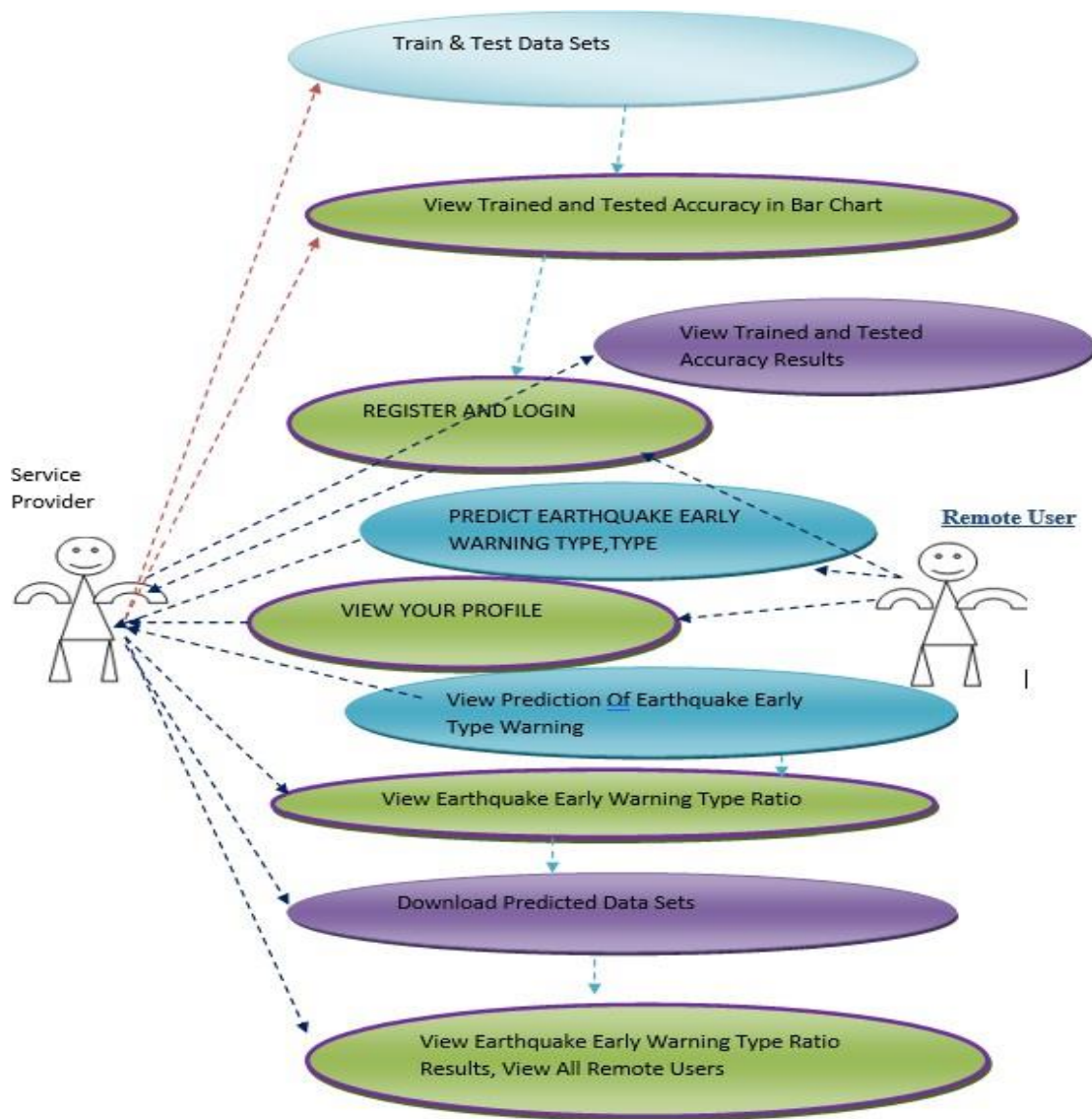


Figure-6.2: Use Case Diagram

Class Diagram :

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram was capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

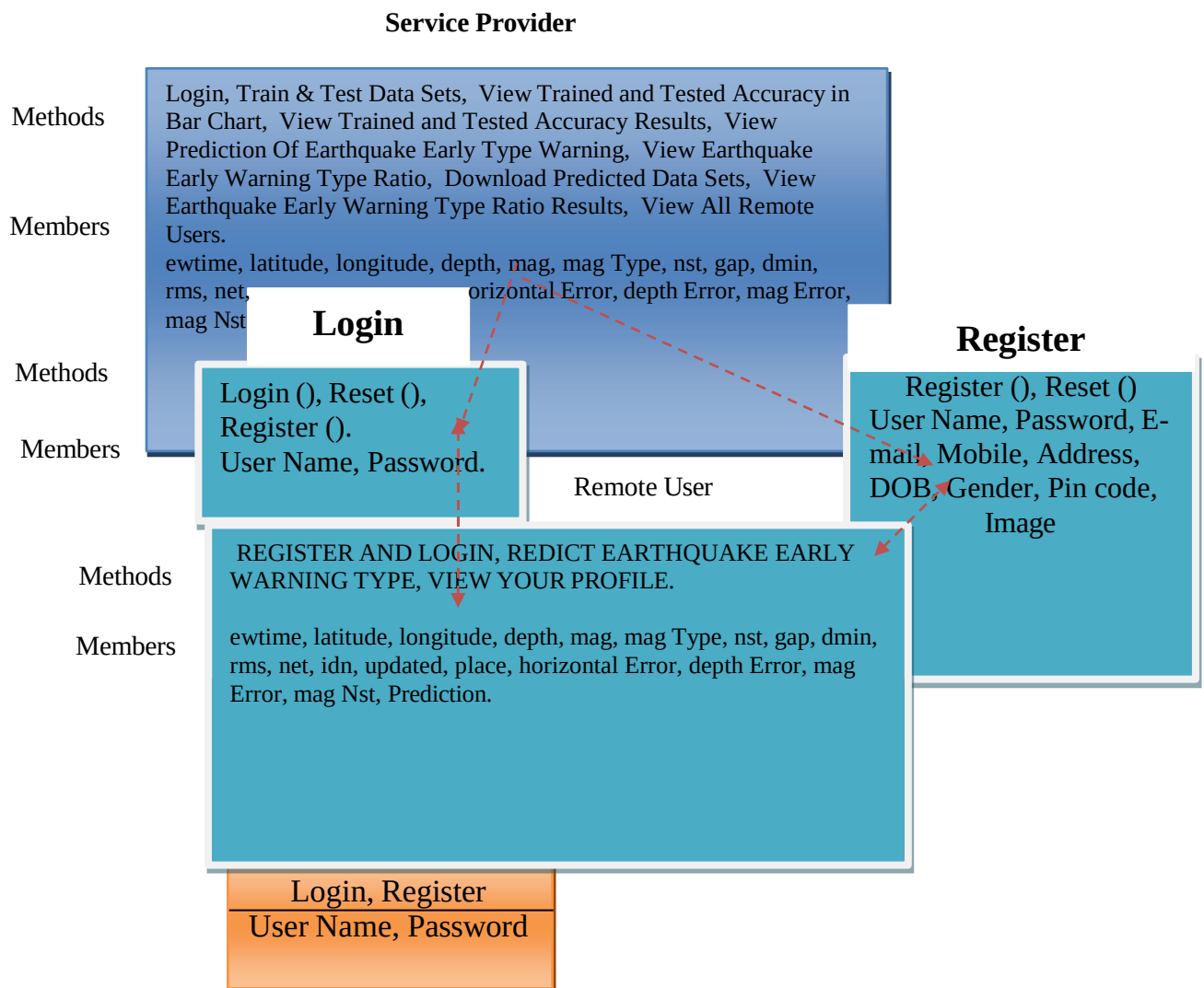


Figure-6.3: Class Diagram

DATA FLOW DIAGRAM:

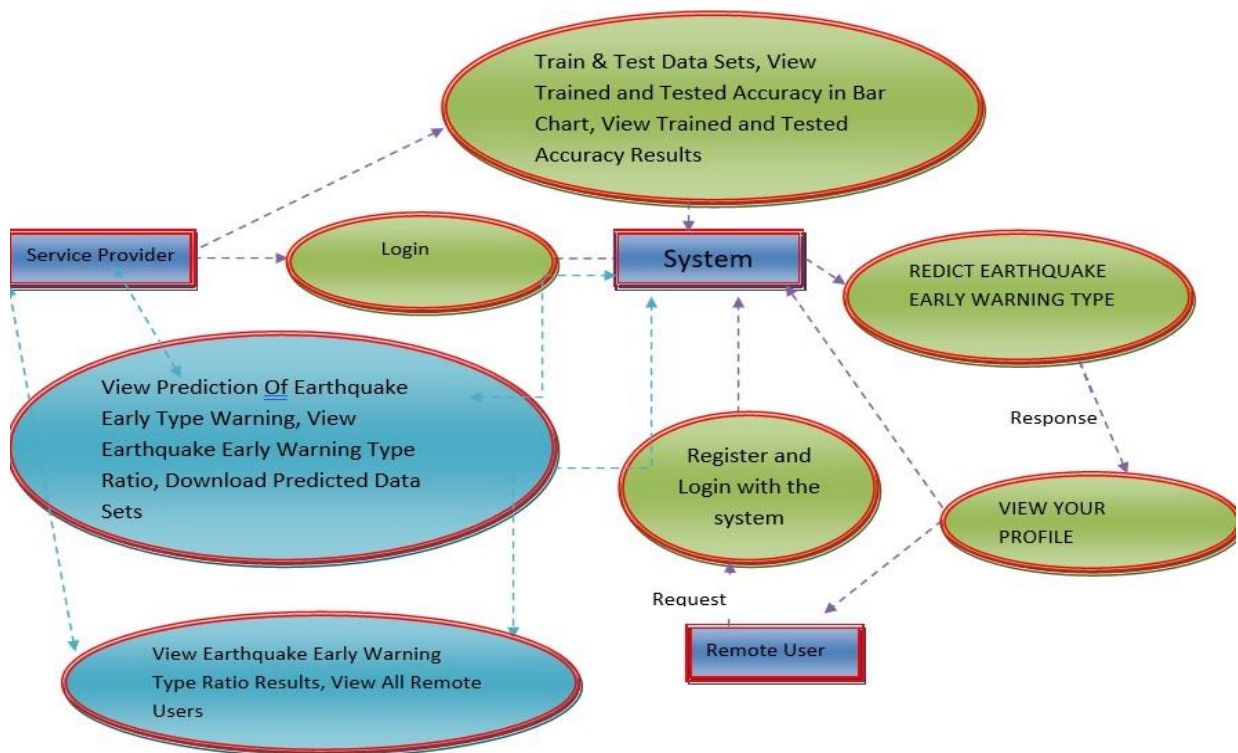


Figure 6.4: Data Flow Diagram

FLOW CHART:- REMOTE USER

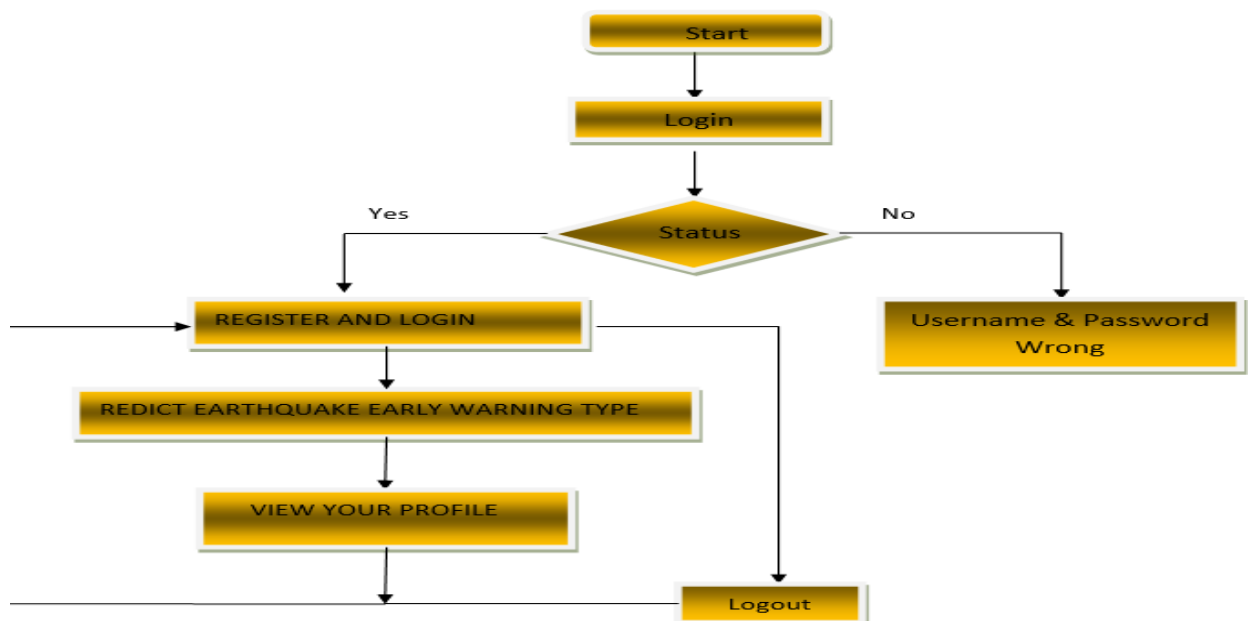


Figure-6.5: Remote User Flow Chart

FLOW CHART:- SERVICE USER

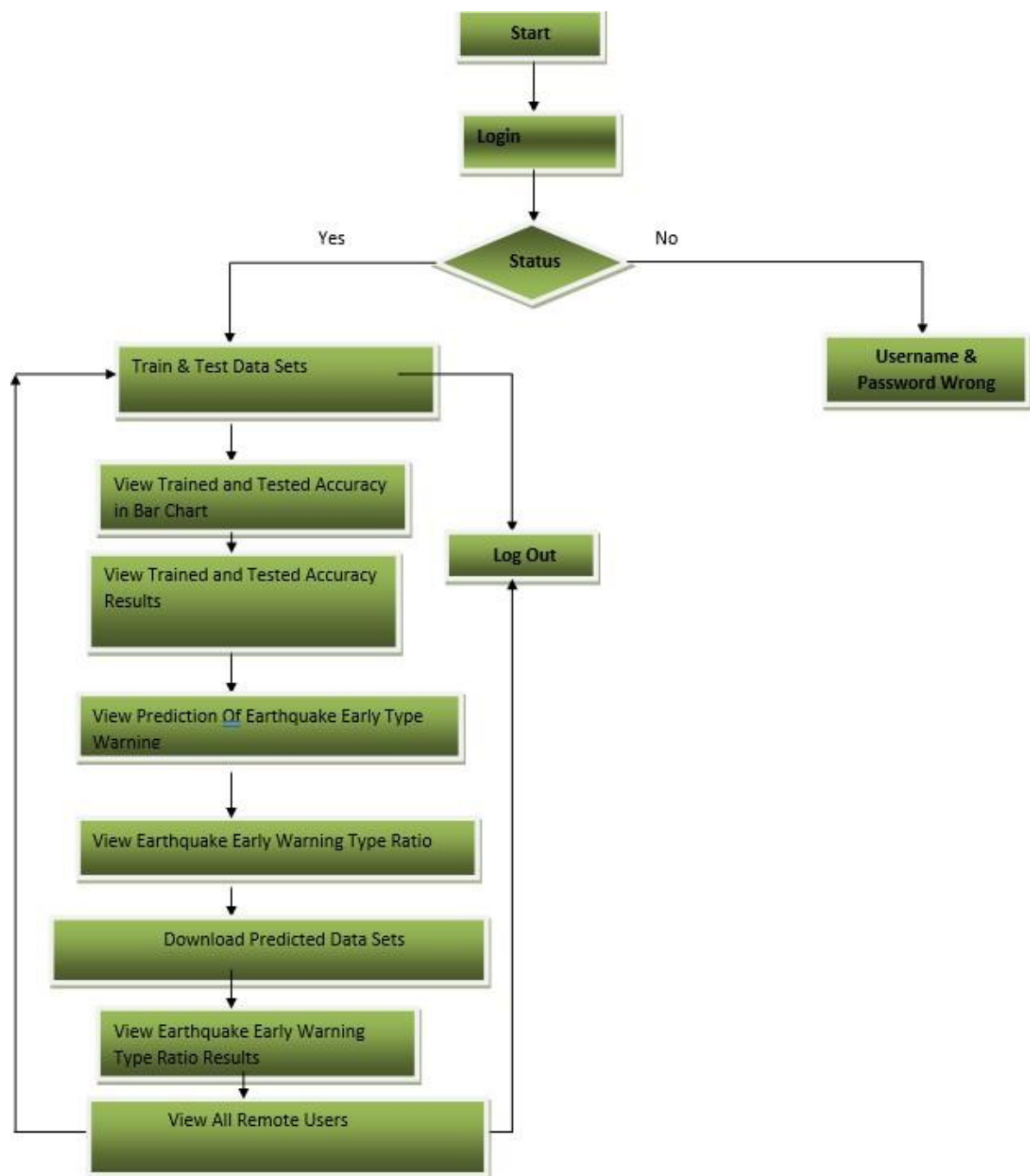


Figure-6.6: Service User Flow Chart

SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

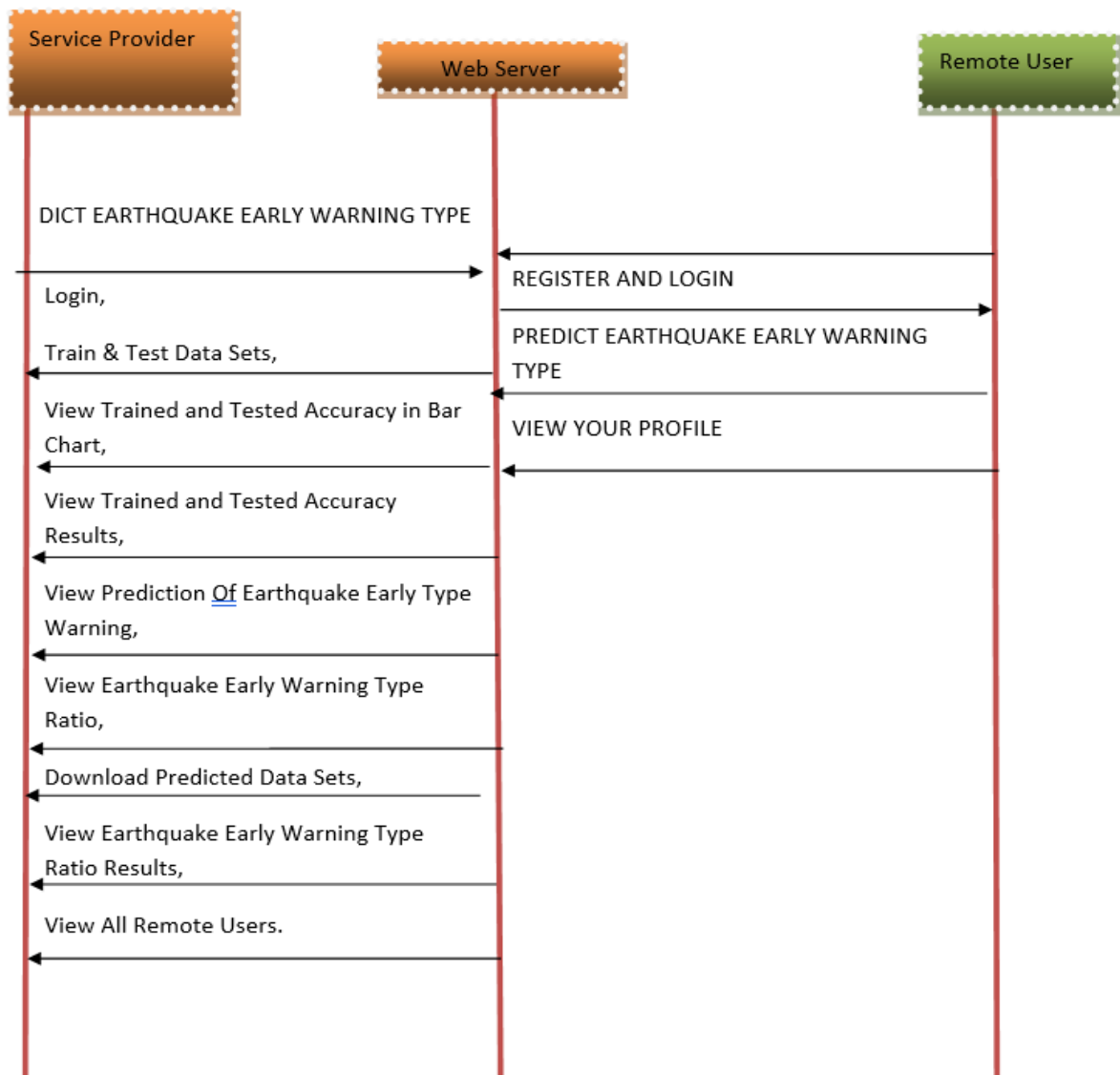


Figure-6.7: Sequence Diagram

CHAPTER-7

SOFTWARE ENVIRONMENT

SOFTWARE ENVIRONMENT

What is python

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Open cv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can use some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print '**Hello World**'. But in Python, just a print statement will do. It is also quite **easy to learn, understand, and code**. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory**. This further aids the readability of the code.

8. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the **encapsulation of data** and functions into one.

9. Free and Open-Source

Like we said earlier, Python is **freely available**. But not only can you [download Python](#) for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with

Python. Here,

you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system- dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

Advantages of Python Over Other Languages :

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machine learning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python:-

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point

for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbannelle**. The reason it is not so famous despite the existence of python is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java DataBase Connectivity)** and **ODBC (Open DataBase Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python : -

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and

to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting

language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain.

Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels

are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction*. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well- defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML –

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term “**Machine Learning**” in 1959 and defined it as a “**Field of study that gives computers the capability to learn without being explicitly programmed**”.

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to [Indeed](#), Machine Learning Engineer Is The Best Job of 2019 with a 344% growth and

an average base salary of
\$146,085 per year.

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is [Python](#)! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as [Keras](#), [TensorFlow](#), [Scikit-learn](#), etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as [Fork Python](#) available Free on Geeks for Geeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on

the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As [ML algorithms](#) gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi- variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

[Machine Learning](#) is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps :-

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt. sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it." Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be

comparable to each other.

- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "/" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers— even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules used

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities
- Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains

including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and [IPython](#) shells, the [Jupyter](#) Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the [sample plots](#) and [thumbnail gallery](#).

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of

development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high- level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. [Download the Python Cheatsheet here](#). The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>



Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab



Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	 Download	Release Notes
Python 3.6.9	July 2, 2019	 Download	Release Notes
Python 3.7.3	March 25, 2019	 Download	Release Notes
Python 3.4.10	March 18, 2019	 Download	Release Notes
Python 3.5.7	March 18, 2019	 Download	Release Notes
Python 2.7.16	March 4, 2019	 Download	Release Notes
Python 3.7.2	Dec. 24, 2018	 Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GP6
Gzipped source tarball	Source release		68111671e5b2db4aef7b9ab01b709be	23017663	3x6
XZ compressed source tarball	Source release		d33e4aa66097051c3eca45ee3604803	17131432	3x6
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.5 and later	6428b4fa7583daf1a442cba0ce09e6	34898436	3x6
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773b5e4a936b2a3f	28082845	3x6
Windows http file	Windows		063999573a2c58b2ac58rade6b4f7nd2	8131761	3x6
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	98093df8dfe0b9a0e03184a40728a2	7504391	3x6
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76d8bdc30c3a5d3e5d3+00	26882368	3x6
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28c31c908b6d72a0e051a3b0301b4bd2	1362904	3x6
Windows x86 embeddable zip file	Windows		9fab3bd19841879fdab412357413bd8	6741626	3x6
Windows x86 executable installer	Windows		33c8822942a5446a3d84c147e394789	25663848	3x6
Windows x86 web-based installer	Windows		1b670cfaed117d82c30953ea371607c	1324608	3x6

To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web- based installer.

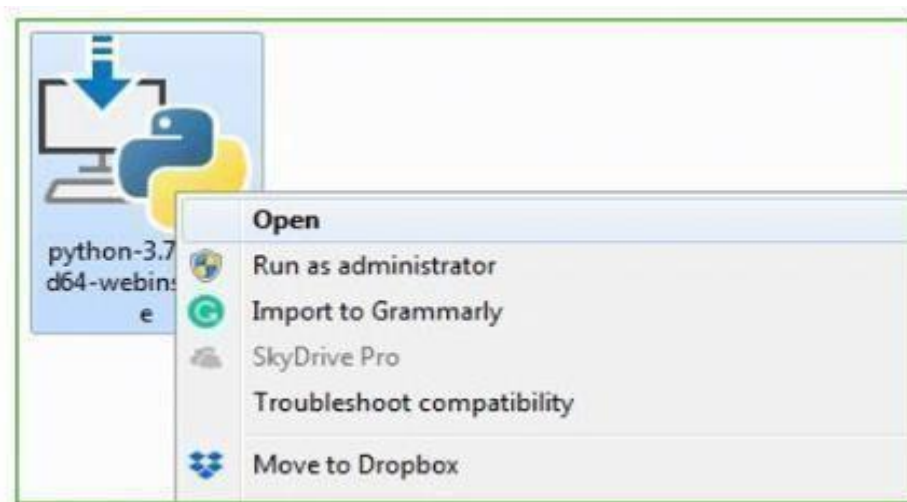
To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web- based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.

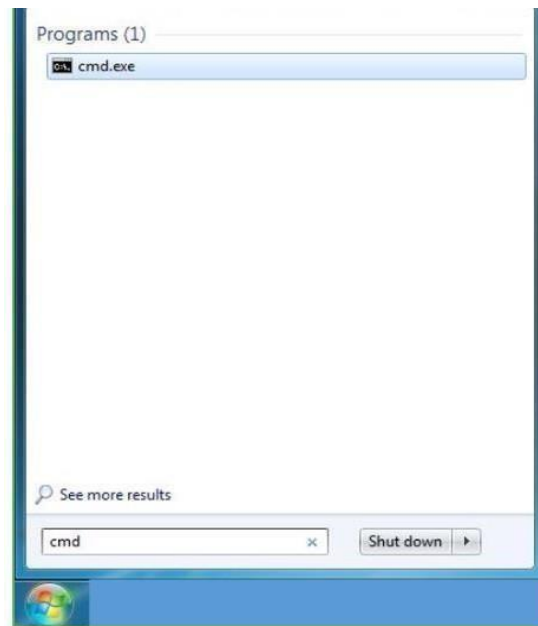


With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes. Verify the Python Installation

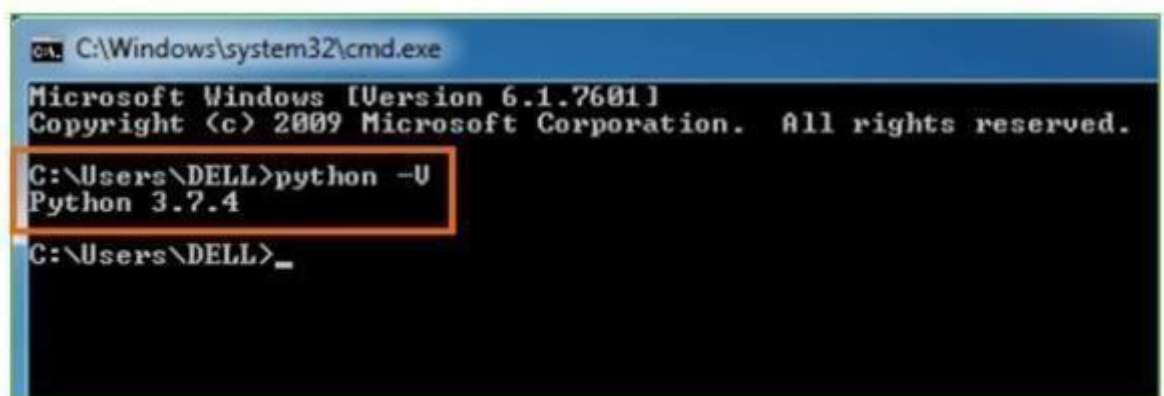
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python -V and press Enter.



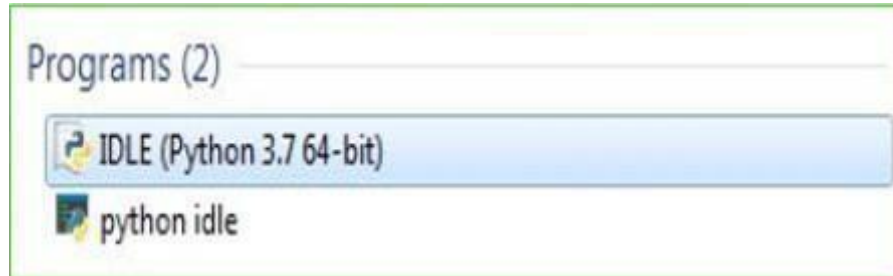
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE

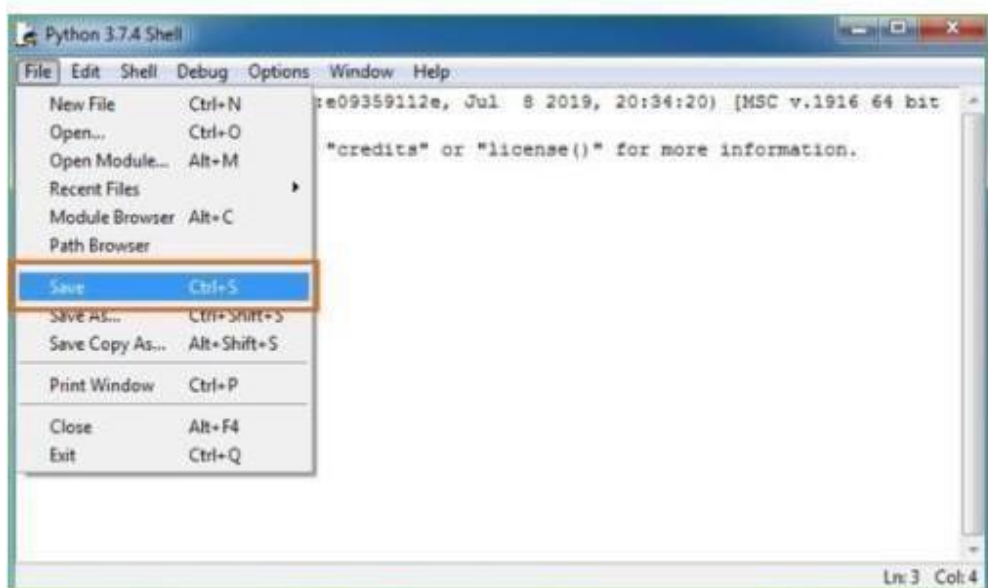
works Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print

CHAPTER-8

SYSTEM TESTING

SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input :identified classes of valid input must be accepted.

Invalid Input :identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output: identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

TESTING METHODOLOGIES

The following are the Testing Methodologies:

- **Unit Testing.**
- **Integration Testing.**
- **User Acceptance Testing.**
- **Output Testing.**
- **Validation Testing.**

Unit Testing

Unit testing focuses verification effort on the smallest unit of Software design that is the module. Unit testing exercises specific paths in a module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit. Hence, the naming is Unit Testing.

During this testing, each module is tested individually and the module interfaces are verified for the consistency with design specification. All important processing path are tested for the expected results. All error handling paths are also tested.

Integration Testing

Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order tests are conducted. The main objective in this testing process is to take unit tested modules and builds a program structure that has been dictated by design.

Using Live Test Data:

Live test data are those that are actually extracted from organization files. After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities. Then, the systems person uses this data as a way to partially test the system. In other instances, programmers or analysts extract a set of live data from the files and have them entered themselves.

It is difficult to obtain live data in sufficient amounts to conduct extensive testing. And, although it is realistic data that will show how the system will perform for the typical processing requirement, assuming that the live data entered are in fact typical, such data generally will not test all combinations or formats that can enter the system. This bias toward typical values then does not provide a true systems test and in fact ignores the cases most likely to cause system failure.

Using Artificial Test Data:

Artificial test data are created solely for test purposes, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information systems department, make possible the testing of all login and control paths through the program.

The most effective test programs use artificial test data generated by persons other than those who wrote the programs. Often, an independent team of testers formulates a testing plan, using the systems specifications.

The package "Virtual Private Network" has satisfied all the requirements specified as per

software requirement specification and was accepted.

USER TRAINING

Whenever a new system is developed, user training is required to educate them about the working of the system so that it can be put to efficient use by those for whom the system has been primarily designed. For this purpose the normal working of the project was demonstrated to the prospective users. Its working is easily understandable and since the expected users are people who have good knowledge of computers, the use of this system is very easy.

OTHER TESTING METHODOLOGIES

User Acceptance Testing

User Acceptance of a system is the key factor for the success of any system. The system under consideration is tested for user acceptance by constantly keeping in touch with the prospective system users at the time of developing and making changes wherever required. The system developed provides a friendly user interface that can easily be understood even by a person who is new to the system.

Output Testing

After performing the validation testing, the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in the specified format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration. Hence the output format is considered in 2 ways – one is on screen and another in printed format.

Validation Checking

Validation checks are performed on the following fields.

Text Field:

The text field can contain only the number of characters lesser than or equal to its size. The text fields are alphanumeric in some tables and alphabetic in other tables. Incorrect entry always flashes and error message.

Numeric Field:

The numeric field can contain only numbers from 0 to 9. An entry of any character flashes an error messages. The individual modules are checked for accuracy and what it has to perform. Each module is subjected to test run along with sample data. The individually tested modules are integrated into a single system. Testing involves executing the real

data information is used in the program the existence of any program defect is inferred from the output. The testing should be planned so that all the requirements are individually tested.

A successful test is one that gives out the defects for the inappropriate data and produces and output revealing the errors in the system.

Preparation of Test Data

Taking various kinds of test data does the above testing. Preparation of test data plays a vital role in the system testing. After preparing the test data the system under study is tested

MAINTAINENCE

This covers a wide range of activities including correcting code and design errors. To reduce the need for maintenance in the long run, we have more accurately defined the user's requirements during the process of system development. Depending on the requirements, this system has been developed to satisfy the needs to the largest possible extent. With development in technology, it may be possible to add many more features based on the requirements in future. The coding and designing is simple and easy to understand which will make maintenance easier.

TESTING STRATEGY :

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation .A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

SYSTEM TESTING:

Software once validated must be combined with other system elements (e.g. Hardware, people, database). System testing verifies that all the elements are proper and that overall system function performance is achieved. It also tests to find discrepancies

between the system and its original objective, current specifications and system documentation.

UNIT TESTING:

In unit testing different modules are tested against the specifications produced during the design for the modules. Unit testing is essential for verification of the code produced during the coding phase, and hence the goal is to test the internal logic of the modules. Using the detailed design description as a guide, important Conrail paths are tested to uncover errors within the boundary of the modules. This testing is carried out during the programming stage itself. In this type of testing step, each module was found to be working satisfactorily as regards to the expected output from the module.

In Due Course, latest technology advancements will be taken into consideration. As part of technical build-up many components of the networking system will be generic in nature so that future projects can either use or interact with this. The future holds a lot to offer to the development and refinement of this project.

CHAPTER-9

SOURCE CODE

SOURCE CODE

VIEWS.PY

```
from django.db.models import Count
from django.db.models import Q
from django.shortcuts import render, redirect, get_object_or_404

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import VotingClassifier
# Create your views here.
from Remote_User.models import
ClientRegister_Model, earthquake_early_warning_prediction, detection_accuracy

def login(request):

    if request.method == "POST" and 'submit1' in request.POST:

        username = request.POST.get('username')
        password = request.POST.get('password')
        try:
            enter = ClientRegister_Model.objects.get(username=username,password=password)
            request.session["userid"] = enter.id

            return redirect('ViewYourProfile')
        except:
            pass

    return render(request, 'RUser/login.html')

def index(request):

    return render(request, 'RUser/index.html')
```

```

def Add_DataSet_Details(request):

    return render(request, 'RUser/Add_DataSet_Details.html', {"excel_data": ""})

def Register1(request):

    if request.method == "POST":
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        phoneno = request.POST.get('phoneno')
        country = request.POST.get('country')
        state = request.POST.get('state')
        city = request.POST.get('city')
        address = request.POST.get('address')
        gender = request.POST.get('gender')
        ClientRegister_Model.objects.create(username=username,
        email=email, password=password, phoneno=phoneno,
        country=country, state=state, city=city,address=address,gender=gender)

        obj = "Registered Successfully"
        return render(request, 'RUser/Register1.html',{'object':obj})
    else:
        return render(request,'RUser/Register1.html')

def ViewYourProfile(request):
    userid = request.session['userid']
    obj = ClientRegister_Model.objects.get(id= userid)
    return render(request,'RUser/ViewYourProfile.html',{'object':obj})

def Predict_Earthquake_Early_Warning_Prediction(request):
    if request.method == "POST":

        if request.method == "POST":

```

```

ewtime= request.POST.get('ewtime')
latitude= request.POST.get('latitude')
longitude= request.POST.get('longitude')
depth= request.POST.get('depth')
mag= request.POST.get('mag')
magType= request.POST.get('magType')
nst= request.POST.get('nst')
gap= request.POST.get('gap')
dmin= request.POST.get('dmin')
rms= request.POST.get('rms')
net= request.POST.get('net')
idn= request.POST.get('idn')
updated1= request.POST.get('updated')
place= request.POST.get('place')
horizontalError= request.POST.get('horizontalError')
depthError= request.POST.get('depthError')
magError= request.POST.get('magError')
magNst= request.POST.get('magNst')

df = pd.read_csv('Earthquake_Warning_Datasets.csv')

def apply_response(Label):
    if (Label == "earthquake warning"):
        return 0 # earthquake
    elif (Label == "No Warning"):
        return 1 # explosion

df['Results'] = df['Label'].apply(apply_response)

cv = CountVectorizer()
X = df['place']
y = df['Results']

print("place")

```

```

print(X)
print("Results")
print(y)

X = cv.fit_transform(X)

models = []
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
X_train.shape, X_test.shape, y_train.shape

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB

NB = MultinomialNB()
NB.fit(X_train, y_train)
predict_nb = NB.predict(X_test)
naivebayes = accuracy_score(y_test, predict_nb) * 100
print("ACCURACY")
print(naivebayes)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_nb))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_nb))
models.append(('naive_bayes', NB))

# SVM Model
print("SVM")
from sklearn import svm

lin_clf = svm.LinearSVC()
lin_clf.fit(X_train, y_train)
predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 100

```

```

print("ACCURACY")
print(svm_acc)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_svm))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
models.append(('svm', lin_clf))

print("Logistic Regression")

from sklearn.linear_model import LogisticRegression

reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
models.append(('logistic', reg))

print("Decision Tree Classifier")
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
dtcpredict = dtc.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, dtcpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, dtcpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, dtcpredict))
models.append(('DecisionTreeClassifier', dtc))

print("Random Forest Classifier")

```



```

from sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier()
rf_clf.fit(X_train, y_train)
rfpredict = rf_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, rfpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, rfpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, rfpredict))
models.append(('RandomForestClassifier', rf_clf))

classifier = VotingClassifier(models)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

place1 = [place]
vector1 = cv.transform(place1).toarray()
predict_text = classifier.predict(vector1)

pred = str(predict_text).replace("[", "")
pred1 = pred.replace("]", "")

prediction = int(pred1)

if(ewtime == "" or ewtime == " "):
    val = "Enter the Details"
elif(prediction == 0):
    val = 'Earthquake Warning'
elif (prediction == 1):
    val = 'No warning'

print(val)
print(pred1)

```

```

earthquake_early_warning_prediction.objects.create(
    ewtime=ewtime,
    latitude=latitude,
    longitude=longitude,
    depth=depth,
    mag=mag,
    magType=magType,
    nst=nst,
    gap=gap,
    dmin=dmin,
    rms=rms,
    net=net,
    idn=idn,
    updated=updated1,
    place=place,
    horizontalError=horizontalError,
    depthError=depthError,
    magError=magError,
    magNst=magNst,
    Prediction=val
)

return render(request, 'RUser/Predict_Earthquake_Early_Warning_Prediction.html',{'objs':
val}))

return render(request, 'RUser/Predict_Earthquake_Early_Warning_Prediction.html')

```

```

from django.db.models import Count, Avg
from django.shortcuts import render, redirect
from django.db.models import Count
from django.db.models import Q
import datetime
import xlwt
from django.http import HttpResponse

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier

# Create your views here.
from Remote_User.models import
ClientRegister_Model, earthquake_early_warning_prediction, detection_ratio,
detection_accuracy
def serviceproviderlogin(request):
    if request.method == "POST":
        admin = request.POST.get('username')
        password = request.POST.get('password')
        if admin == "Admin" and password == "Admin":
            detection_accuracy.objects.all().delete()
            return redirect('View_Remote_Users')

    return render(request, 'SProvider/serviceproviderlogin.html')

def View_Prediction_Of_Earthquake_Early_Warning_Ratio(request):
    detection_ratio.objects.all().delete()
    ratio = ""
    kword = 'Earthquake Warning'
    print(kword)
    obj = earthquake_early_warning_prediction.objects.all().filter(Q(Prediction=kword))

```

```

obj1 = earthquake_early_warning_prediction.objects.all()
count = obj.count();
count1 = obj1.count();
ratio = (count / count1) * 100
if ratio != 0:
    detection_ratio.objects.create(names=kword, ratio=ratio)

ratio12 = ""
kword12 = 'No Warning'
print(kword12)
obj12 = earthquake_early_warning_prediction.objects.all().filter(Q(Prediction=kword12))
obj112 = earthquake_early_warning_prediction.objects.all()
count12 = obj12.count();
count112 = obj112.count();
ratio12 = (count12 / count112) * 100
if ratio12 != 0:
    detection_ratio.objects.create(names=kword12, ratio=ratio12)

obj = detection_ratio.objects.all()
return render(request, 'SProvider/View_Prediction_Of_Earthquake_Early_Warning_
Ratio.html', {'objs': obj})

def View_Remote_Users(request):
    obj=ClientRegister_Model.objects.all()
    return render(request,'SProvider/View_Remote_Users.html',{'objects':obj})

def charts(request,chart_type):
    chart1 = detection_ratio.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"SProvider/charts.html", {'form':chart1, 'chart_type':chart_type})

def charts1(request,chart_type):
    chart1 = detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"SProvider/charts1.html", {'form':chart1, 'chart_type':chart_type})

def View_Prediction_Of_Earthquake_Early_Warning(request):

```

```

    obj = earthquake_early_warning_prediction.objects.all()
    return render(request, 'SProvider/View_Prediction_Of_Earthquake_
Early_Warning.html', {'list_objects': obj})

def likeschart(request, like_chart):
    charts = detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request, "SProvider/likeschart.html",
{'form': charts, 'like_chart': like_chart})

def Download_Trained_DataSets(request):

    response = HttpResponse(content_type='application/ms-excel')
    # decide file name
    response['Content-Disposition'] = 'attachment; filename="Predicted_Datasets.xls"'
    # creating workbook
    wb = xlwt.Workbook(encoding='utf-8')
    # adding sheet
    ws = wb.add_sheet("sheet1")
    # Sheet header, first row
    row_num = 0
    font_style = xlwt.XFStyle()
    # headers are bold
    font_style.font.bold = True
    # writer = csv.writer(response)
    obj = earthquake_early_warning_prediction.objects.all()
    data = obj # dummy method to fetch data.
    for my_row in data:
        row_num = row_num + 1

        ws.write(row_num, 0, my_row.ewtime, font_style)
        ws.write(row_num, 1, my_row.latitude, font_style)
        ws.write(row_num, 2, my_row.longitude, font_style)
        ws.write(row_num, 3, my_row.depth, font_style)
        ws.write(row_num, 4, my_row.mag, font_style)
        ws.write(row_num, 5, my_row.magType, font_style)

```

```
ws.write(row_num, 6, my_row.nst, font_style)
ws.write(row_num, 7, my_row.gap, font_style)
ws.write(row_num, 8, my_row.dmin, font_style)
ws.write(row_num, 9, my_row.rms, font_style)
ws.write(row_num, 10, my_row.net, font_style)
ws.write(row_num, 11, my_row.idn, font_style)
ws.write(row_num, 12, my_row.updated, font_style)
ws.write(row_num, 13, my_row.place, font_style)
ws.write(row_num, 14, my_row.horizontalError, font_style)
ws.write(row_num, 15, my_row.depthError, font_style)
ws.write(row_num, 16, my_row.magError, font_style)
ws.write(row_num, 17, my_row.magNst, font_style)
ws.write(row_num, 18, my_row.Prediction, font_style)
```

```
wb.save(response)
```

```
return response
```

```
def train_model(request):
```

```
    detection_accuracy.objects.all().delete()
```

```
df = pd.read_csv('Earthquake_Warning_Datasets.csv')
```

```
def apply_response(Label):
```

```
    if (Label == "earthquake warning"):
```

```
        return 0 # earthquake
```

```
    elif (Label == "No Warning"):
```

```
        return 1 # explosion
```

```
df['Results'] = df['Label'].apply(apply_response)
```

```
cv = CountVectorizer()
```

```
X = df['place']
```

```
y = df['Results']
```

```
print("place")
```

```

print(X)
print("Results")
print(y)

X = cv.fit_transform(X)

models = []
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
X_train.shape, X_test.shape, y_train.shape

print(X_test)

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB
NB = MultinomialNB()
NB.fit(X_train, y_train)
predict_nb = NB.predict(X_test)
naivebayes = accuracy_score(y_test, predict_nb) * 100
print(naivebayes)
print(confusion_matrix(y_test, predict_nb))
print(classification_report(y_test, predict_nb))
models.append(('naive_bayes', NB))
detection_accuracy.objects.create(names="Naive Bayes", ratio=naivebayes)

# SVM Model
print("SVM")
from sklearn import svm
lin_clf = svm.LinearSVC()
lin_clf.fit(X_train, y_train)
predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 100
print(svm_acc)
print("CLASSIFICATION REPORT")

```

```

print(classification_report(y_test, predict_svm))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
models.append(('svm', lin_clf))
detection_accuracy.objects.create(names="SVM", ratio=svm_acc)

print("Logistic Regression")

from sklearn.linear_model import LogisticRegression
reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
models.append(('logistic', reg))
detection_accuracy.objects.create(names="Logistic Regression",
    ratio=accuracy_score(y_test, y_pred) * 100)

print("Decision Tree Classifier")
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
dtcpredict = dtc.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, dtcpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, dtcpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, dtcpredict))
models.append(('DecisionTreeClassifier', dtc))
detection_accuracy.objects.create(names="Decision Tree Classifier",
    ratio=accuracy_score(y_test, dtcpredict) * 100)

```



```

print("KNeighborsClassifier")

from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier()
kn.fit(X_train, y_train)
knpredict = kn.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, knpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, knpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, knpredict))
detection_accuracy.objects.create(names="KNeighborsClassifier",
ratio=accuracy_score(y_test, knpredict) * 100)

print("Random Forest Classifier")
from sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier()
rf_clf.fit(X_train, y_train)
rfpredict = rf_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, rfpredict) * 100)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, rfpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, rfpredict))
models.append(('RandomForestClassifier', rf_clf))
    detection_accuracy.objects.create(names="RandomForestClassifier",
    ratio=accuracy_score(y_test, rfpredict) * 100)
csv_format = 'Results.csv'
df.to_csv(csv_format, index=False)
df.to_markdown

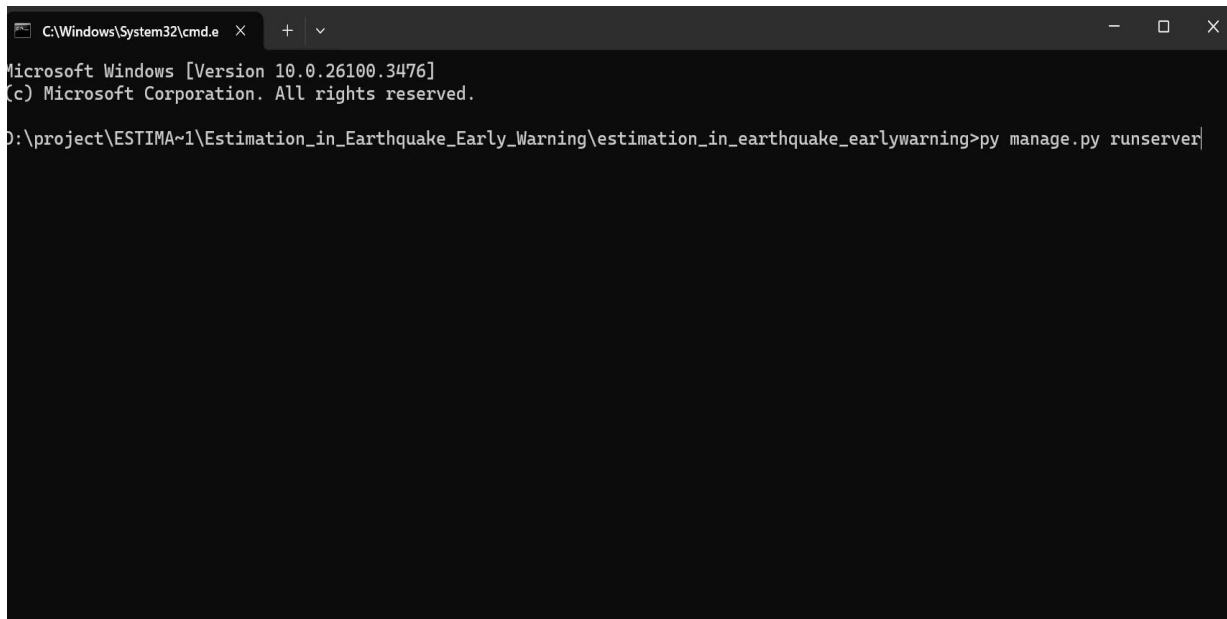
obj = detection_accuracy.objects.all()
return render(request, 'SProvider/train_model.html', {'objs': obj})

```

CHAPTER-10

OUTPUTS

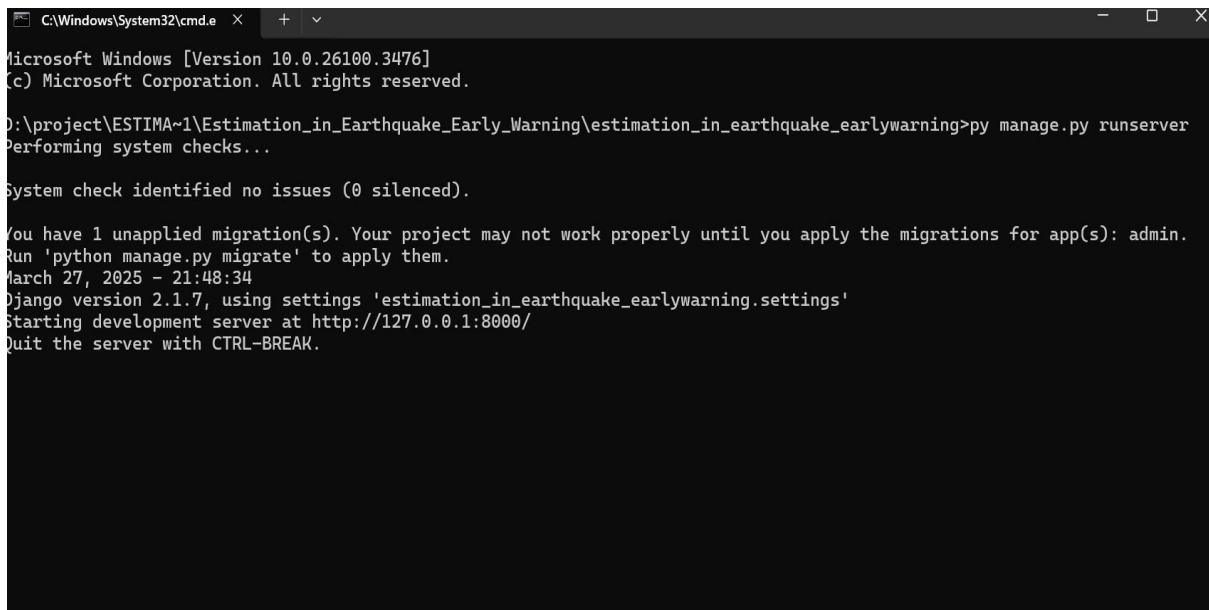
OUTPUTS



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

D:\project\ESTIMA~1\Estimation_in_Earthquake_Early_Warning\estimation_in_earthquake_earlywarning>py manage.py runserver
```

Fig 10.1: Starting command to run the Django application” python manage.py runserver”.



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

D:\project\ESTIMA~1\Estimation_in_Earthquake_Early_Warning\estimation_in_earthquake_earlywarning>py manage.py runserver
Performing system checks...

System check identified no issues (0 silenced).

You have 1 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin.
Run 'python manage.py migrate' to apply them.
March 27, 2025 - 21:48:34
Django version 2.1.7, using settings 'estimation_in_earthquake_earlywarning.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Fig 10.2: generation of Hyperlink.

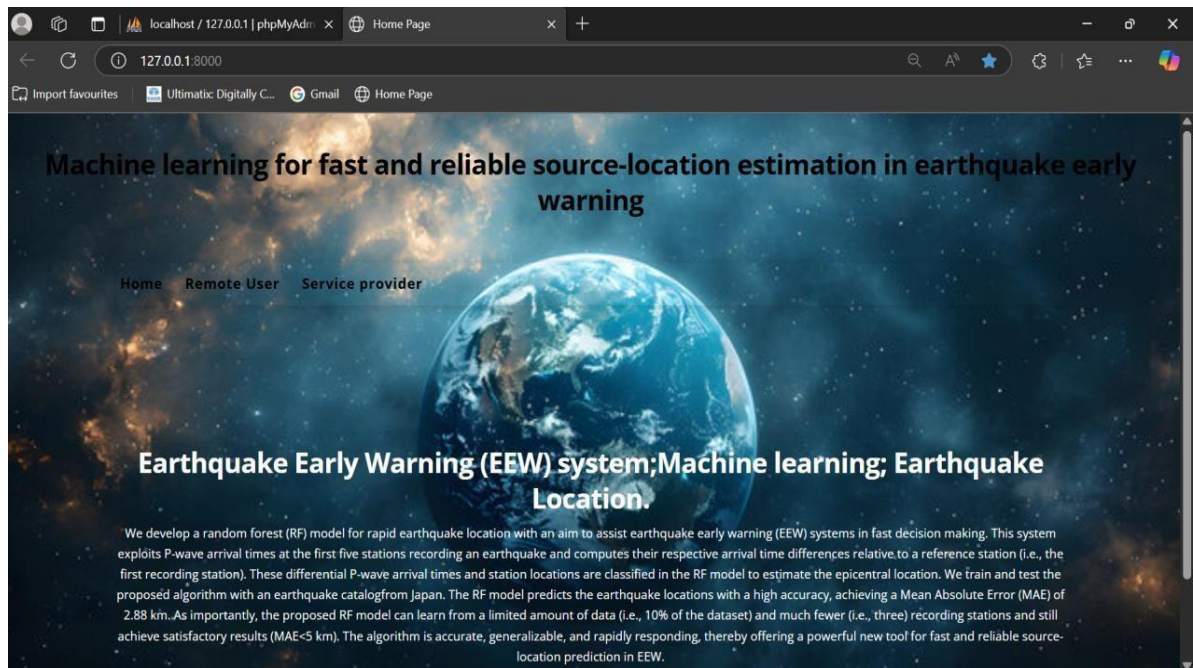


Fig 10.3: Home.html

Fig 10.4: If You want to login in to the page new user must be registered .

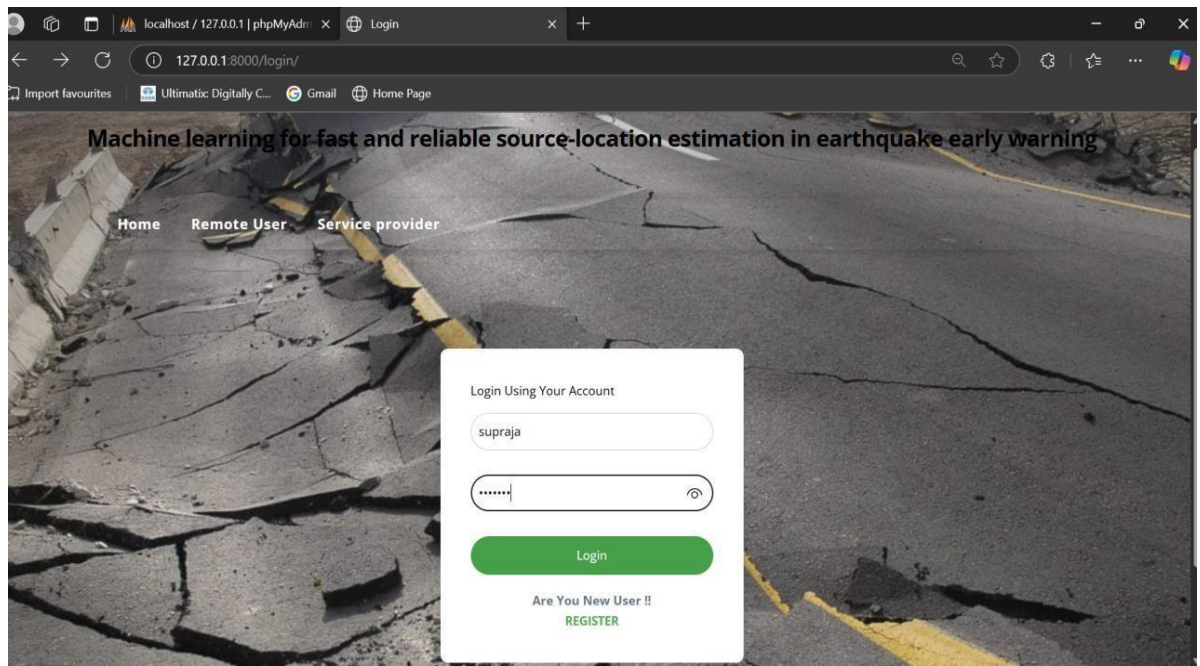


Fig 10.5: Login to remote user page

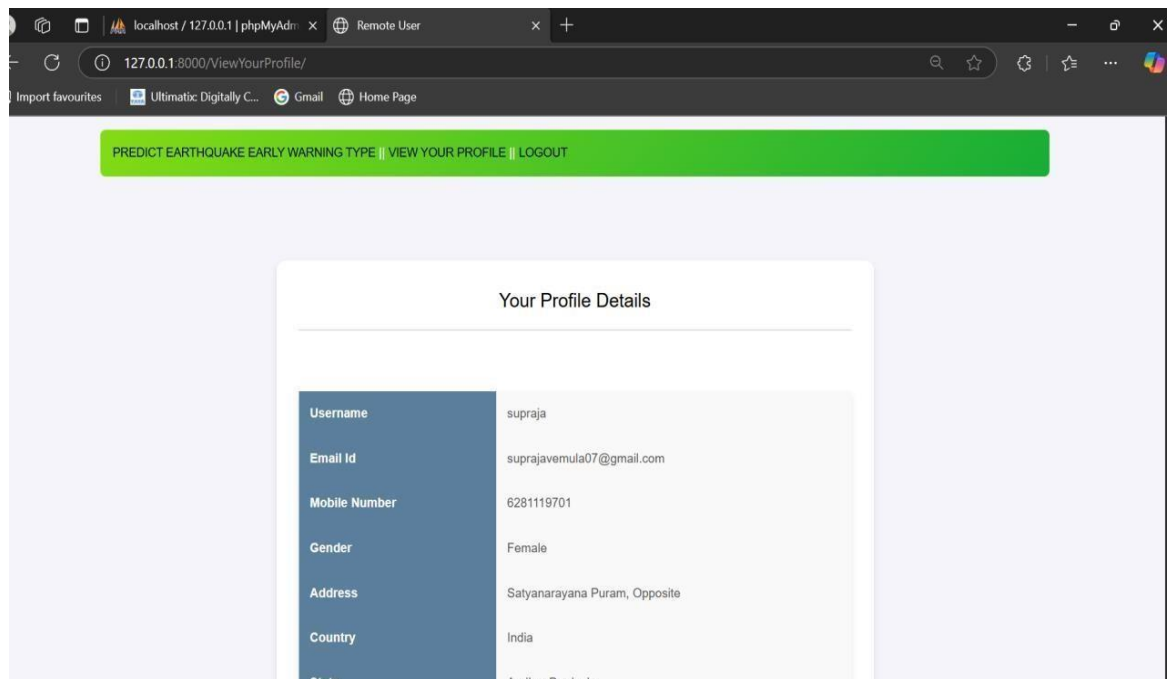


Fig 10.6: User can view their details in profile.

Machine learning for fast and reliable source-location estimation in earthquake early warning

PREDICT EARTHQUAKE EARLY WARNING TYPE || VIEW YOUR PROFILE || LOGOUT

PREDICTION OF EARTHQUAKE EARLY WARNING !!!

Enter Datasets Details Here !!!

Enter ewtime		Enter latitude	
Enter longitude		Enter depth	
Enter mag		Enter magType	

Enter nst		Enter gap	
Enter dmin		Enter rms	
Enter net		Enter idn	
Enter updated Time		Enter place	
Enter horizontalError		Enter depthError	
Enter magError		Enter magNst	

Predict

Fig 10.7 : Fill the details for prediction of early earthquake warning and then predict the output.

Prediction Of Earthquake Early Warning

No warning

Fig 10.8 : Output will be predicted like this.

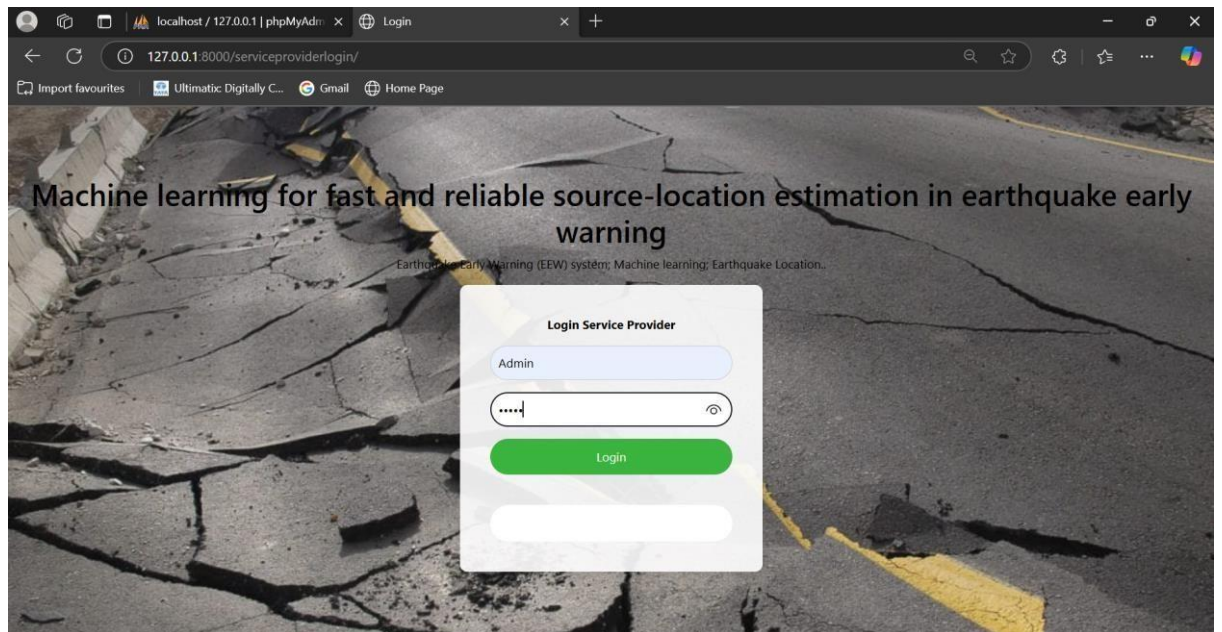


Fig 10.9 : Admin will be login into service provider page.

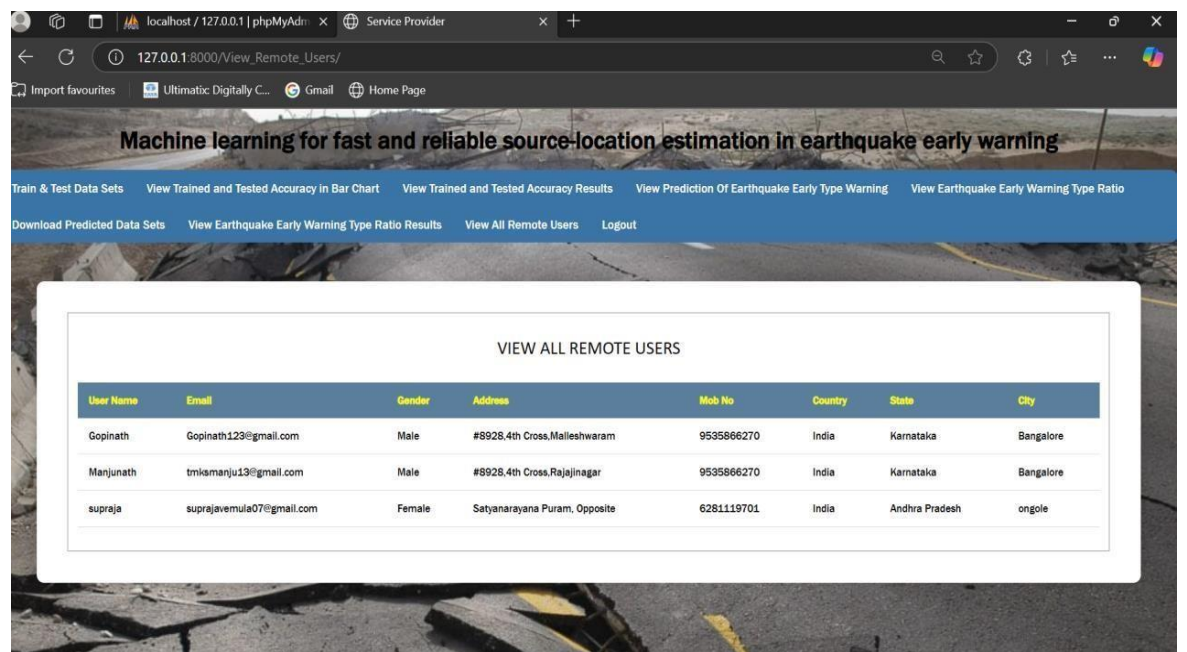


Fig 10.10 : we can view all remote user here.

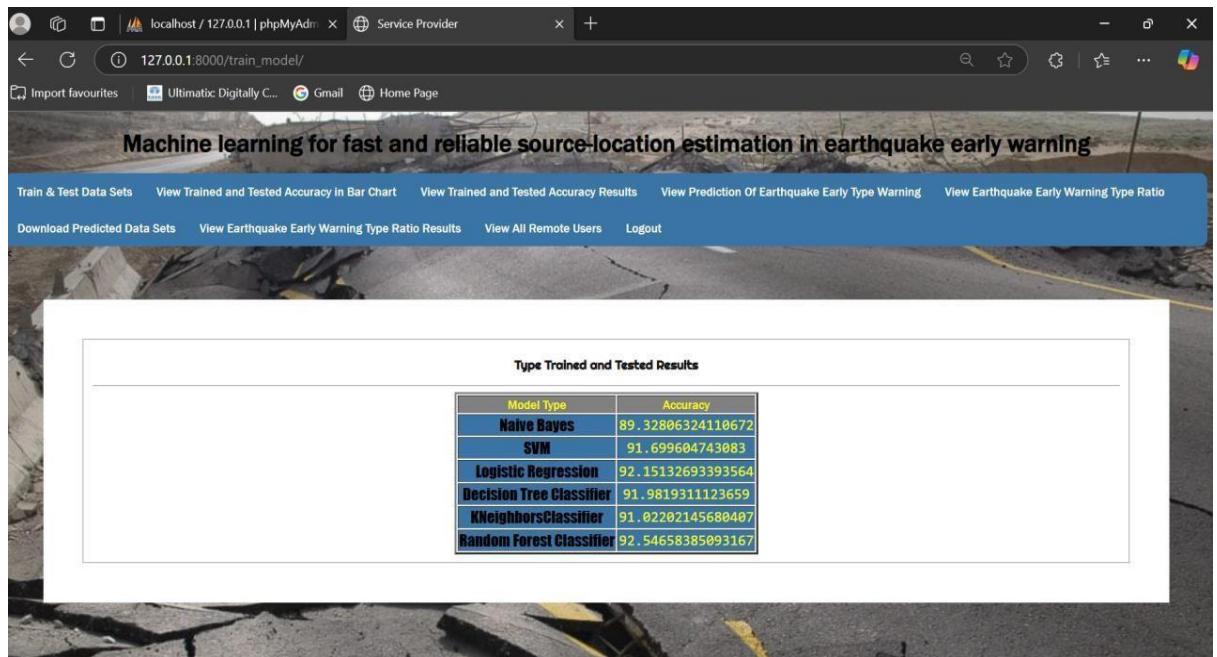


Fig 10.11 : Here dataset will be trained and tested.

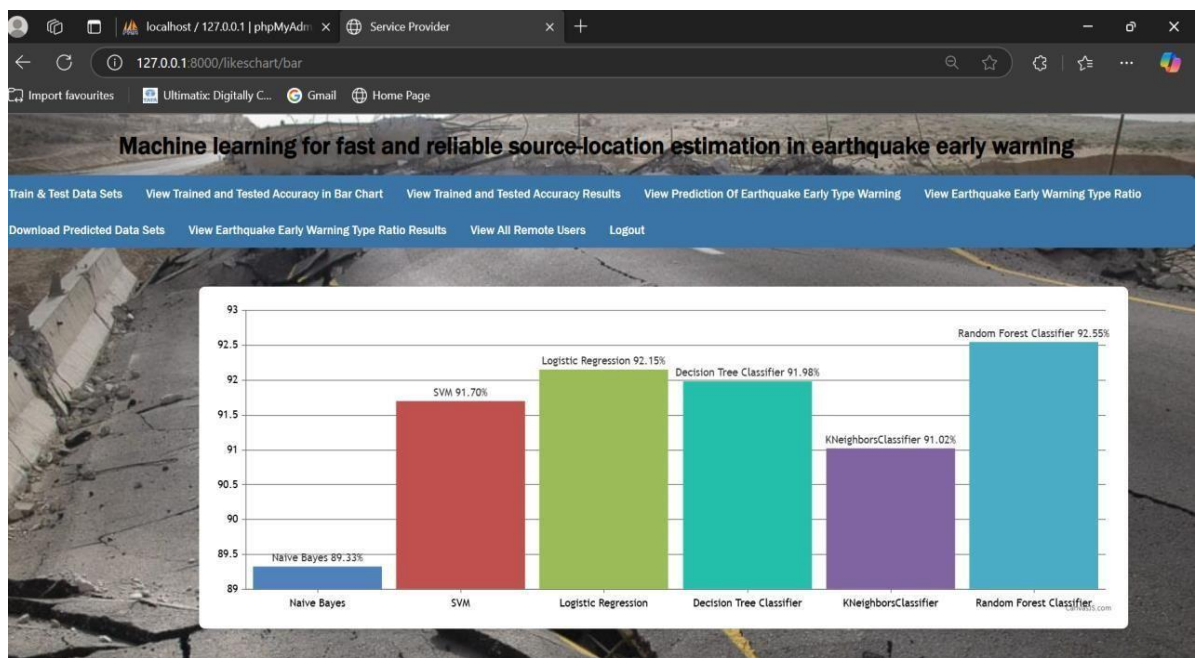


Fig 10.12 : Here dataset can be represented in a bar chart.

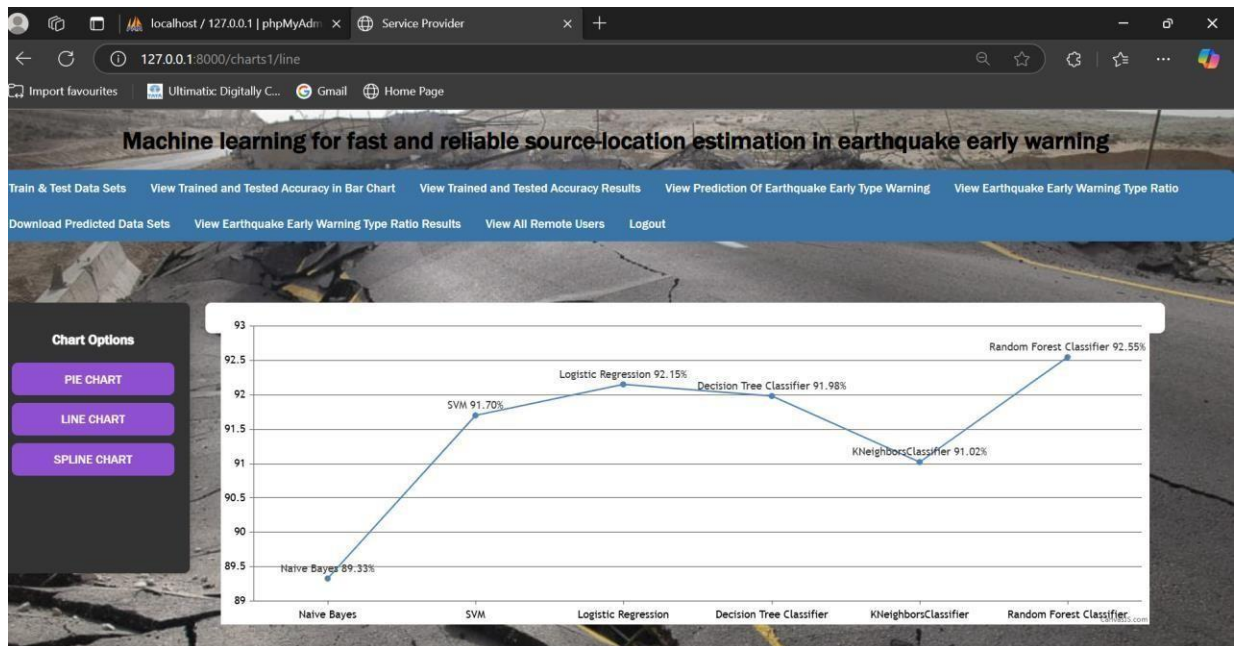


Fig 10.13 : Here data accuracy results will be represented in different chart options.

ewtime	latitude	longitude	depth	mag	magType	nst	gap	dmin	rms	net	idn
2022-08-23T02:02:56.400Z	19.18916702	-155.5106659	35.77999878	2.2	ml	47	90	NA	0.1099999999	hv	hv73118182
2022-08-23T00:35:27.226Z	-52.5504	-71.7165	35.032	4.2	mb	26	150	0.32	0.72	us	us6000icx6
2022-08-22T22:27:15.485Z	58.4812	-154.8813	10	1	ml	NA	NA	NA	1.04	ak	ak022ard0ke4
2022-08-23T00:35:27.226Z	38.8326683	-122.8191681	1.38	0.83	ml	21	57	0.01167	0.05	nc	nc7370790

Fig 10.14 : Here you can view prediction of earthquake early type warning.

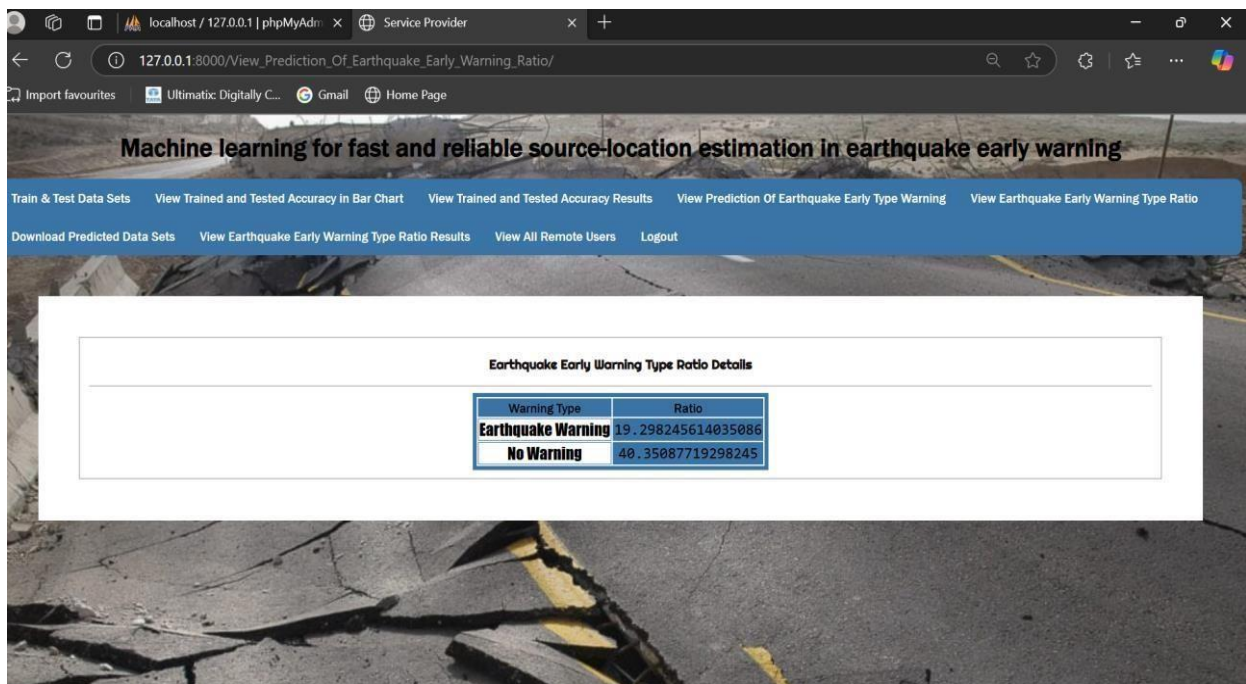


Fig 10.15 : view earthquake early warning type ratio.

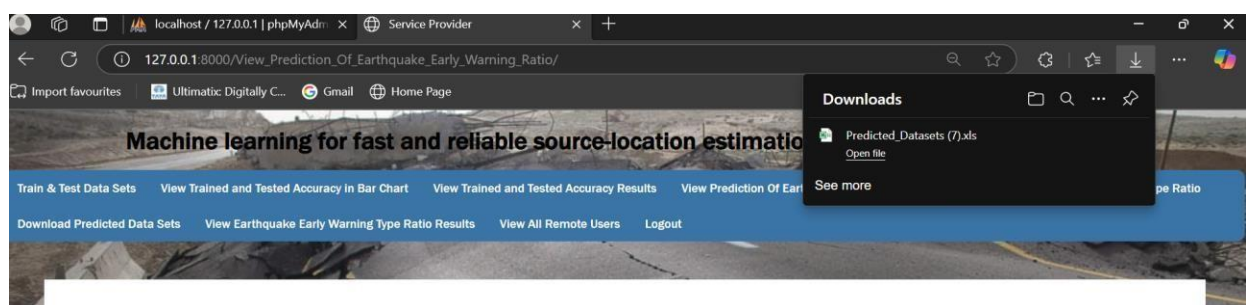


Fig 10.16 : Download datasets.

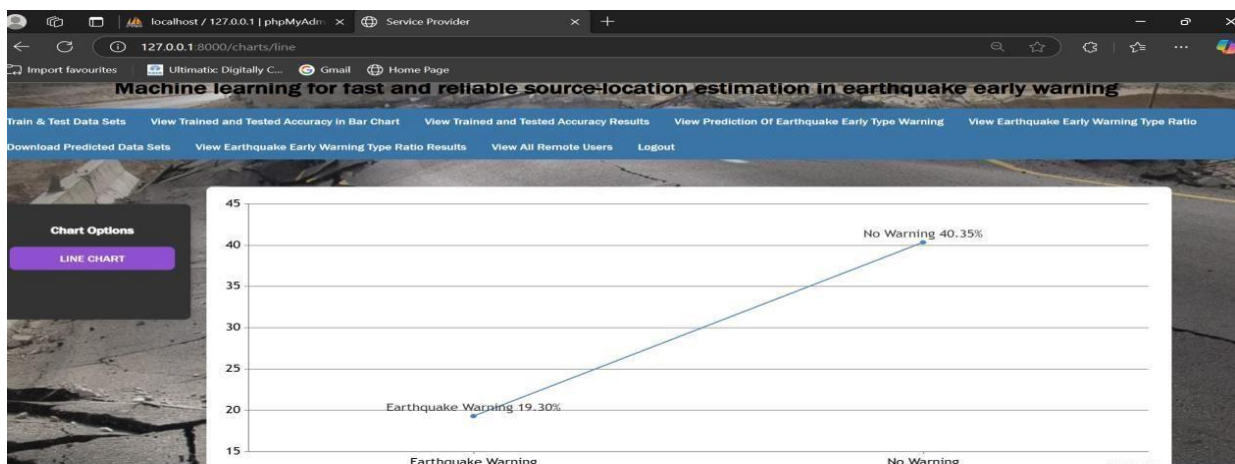


Fig 10.17 : Ratio results will be represented in chart form.

CHAPTER-11

CONCLUSION

CONCLUSION

We use the P-wave arrival time differences and the location of the seismic stations to locate the earthquake in a real-time way. Random forest (RF) has been proposed to perform this regression problem, where the difference latitude and longitude between the earthquake and the seismic stations are considered as the RF output. The Japanese seismic area is used as a case of study, which demonstrates very successful performance and indicates its immediate applicability.

CHAPTER-12

BIBLIOGRAPY

BIBLIOGRAPHY

- [1] Q. Kong, R. M. Allen, L. Schreier, and Y.-W. Kwon, "Myshake: A smartphone seismic network for earthquake early warning and beyond," *Science advances*, vol. 2, no. 2, p. e1501055, 2016.
- [2] T.-L. Chin, K.-Y. Chen, D.-Y. Chen, and D.-E. Lin, "Intelligent real-time earthquake detection by recurrent neural networks," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 58, no. 8, pp. 5440–5449, 2020.
- [3] T.-L. Chin, C.-Y. Huang, S.-H. Shen, Y.-C. Tsai, Y. H. Hu, and Y.-M. Wu, "Learn to detect: Improving the accuracy of earthquake detection," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 57, no. 11, pp. 8867–8878, 2019.
- [4] O. M. Saad, A. G. Hafez, and M. S. Soliman, "Deep learning approach for earthquake parameters classification in earthquake early warning system," *IEEE Geoscience and Remote Sensing Letters*, pp. 1–5, 2020.
- [5] X. Zhang, J. Zhang, C. Yuan, S. Liu, Z. Chen, and W. Li, "Locating induced earthquakes with a network of seismic stations in Oklahoma via a deep learning method," *Scientific reports*, vol. 10, no. 1, pp. 1–12, 2020.
- [6] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] S. M. Mousavi, W. L. Ellsworth, W. Zhu, L. Y. Chuang, and G. C. Beroza, "Earthquake transformer: an attentive deep-learning model for simultaneous earthquake detection and phase picking," *Nature Communications*, vol. 11, no. 1, pp. 1–12, 2020.
- [8] S. M. Mousavi and G. C. Beroza, "A Machine-Learning Approach for Earthquake Magnitude Estimation," *Geophysical Research Letters*, vol. 47, no. 1, p. e2019GL085976, 2020.